

AgriDabaw-3D

System Architecture and Code Reference

How the Unity game and the Spring Boot server fit together, what every script and file does, and how the new planting system works.

September 2026 · Unity client (AgriDabao2ndVer) and backend (agridabao-api)

NEW · planting file created for the planting system

CHANGED · planting file updated for the planting system

Contents

Part 1 — The system at a glance	4
How to read this document	4
1.1 The architecture at a glance	4
1.2 The player's journey through the three scenes	5
1.3 Design patterns used across the Unity code	5
1.4 Game time and the simulation loop	6
1.5 The main data flows	6
Part 2 — The planting system	8
2.1 What the planting system changed	8
2.2 Per-crop rules	8
2.3 How it works technically	9
Part 3 — Unity game: every script	12
3.1 Area selection: district data	12
3.2 Area selection: the screen	12
3.3 Main menu and accounts	13
3.4 Shared services (used in every scene)	14
3.5 AI farm adviser (Antonio chat)	17
3.6 Farm audio	18
3.7 Climate event evaluation	18
3.8 Crop care and climate structures	18
3.9 Crop simulation	20
3.10 Developer and test tools	21
3.11 Shop, prices and selling	21
3.12 Farming interactions	22
3.13 Daily tasks and AI tasks	23
3.14 Harvested produce	24
3.15 Inventory	25
3.16 Farm map and boundary	25
3.17 Seedling Tent and planting materials (new folder)	26
3.18 Saving and loading the farm	27
3.19 Pests and diseases	27
3.20 Player control	28
3.21 Friends, chat, marketplace and trades	29
3.22 Soil	30
3.23 Terrain and world	30
3.24 Beginner tutorial (Antonio)	31
3.25 Farm popups and loading	32
3.26 Time and weather	32
3.27 Editor tools (run inside Unity, not in the game)	32
3.28 Other project folders	33
Part 4 — Backend server: every file	34
4.1 Application entry point	34
4.2 Security, tokens and error handling	34
4.3 Sign-up, login and password recovery	35

4.4 The account	36
4.5 Farm saves and the economy	36
4.6 Marketplace	38
4.7 Player-to-player trades	38
4.8 Friends, presence and chat	38
4.9 AI proxy for the farm adviser	39
4.10 Player settings	39
4.11 Email	39
4.12 Developer and admin tools	40
4.13 Tests	40
Part 5 — Database, configuration and deployment	41
5.1 Database tables (Flyway migrations V1–V11)	41
5.2 Configuration	41
5.3 Build and deployment files	42
Part 6 — Questions a panel may ask	43

Part 1 — The system at a glance

What the two halves are, how they talk, and the ideas that run through the code.

How to read this document

The system has two halves that are built, deployed and explained separately:

- **The game:** a Unity 6 project written in C#. It runs on the player's Android phone. It draws the 3D farm, simulates the crops, weather and pests, and builds every screen in code.
- **The backend:** a Spring Boot web API written in Java. It runs on Railway with a PostgreSQL database. It owns accounts, the saved farms, the marketplace, trades, chat and the AI proxy.

Part 1 explains how the two halves fit together. **Part 2** explains the new planting system in depth. **Part 3** lists every Unity script and **Part 4** every backend file, grouped by folder, each with what it does and its most important parts. **Part 5** covers the database, configuration and deployment. The last section is a list of questions a panel is likely to ask, with the file that answers each.

Badges beside a file name: **NEW · planting** the file was created for the planting system; **CHANGED · planting** the file existed and was updated for it. File names in `code style` are exact. "Key parts" names the classes and methods worth pointing at when explaining the file.

1.1 The architecture at a glance

The phone talks to the backend over HTTPS using JSON (a REST API). After signing in, every request carries a **JWT access token** in the `Authorization: Bearer ...` header. The server checks the token before doing anything. The phone never talks to the database or to Google Gemini directly. Only the soil lookup goes straight from the phone to an outside service (ISRIC SoilGrids).

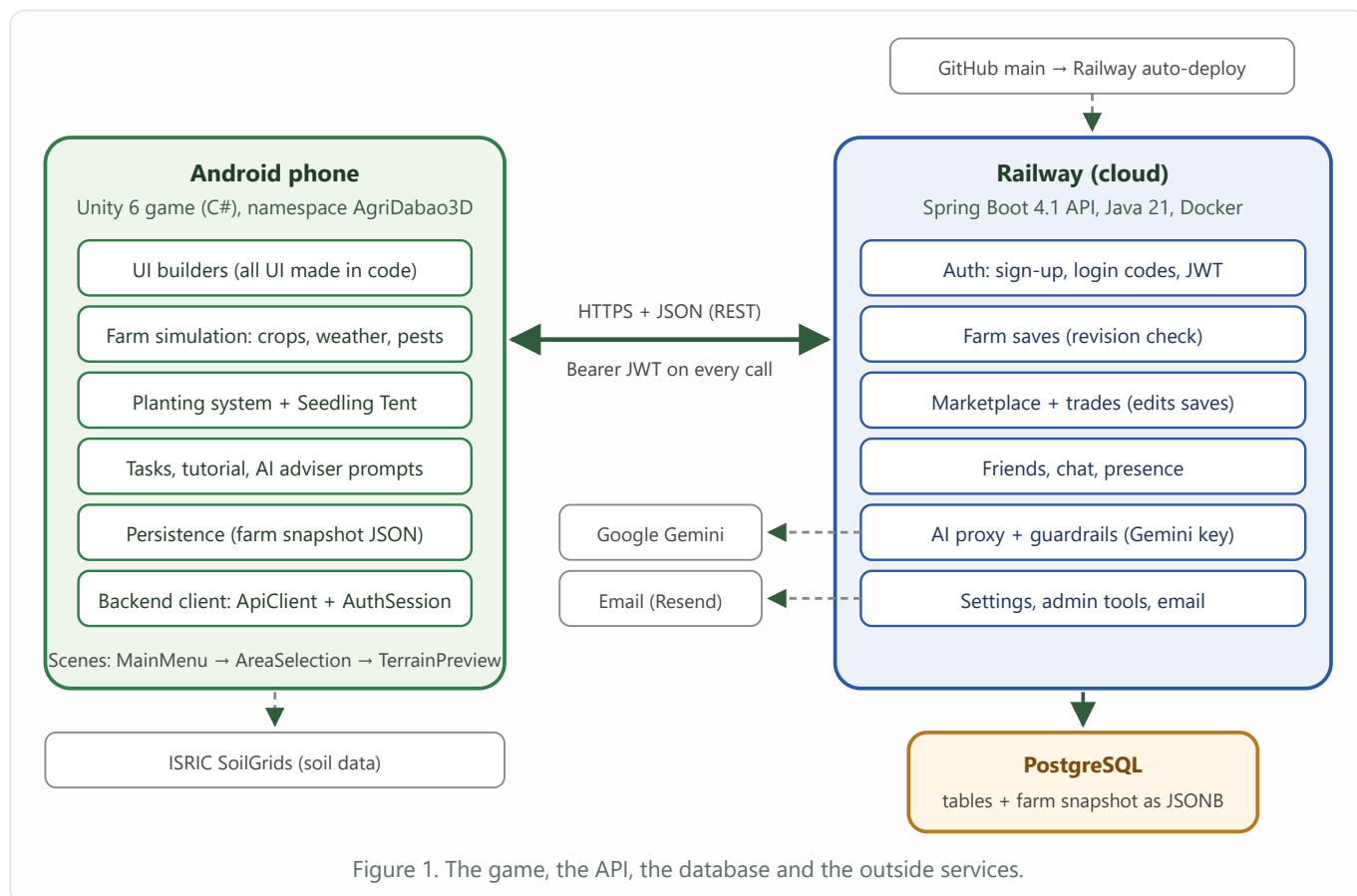


Figure 1. The game, the API, the database and the outside services.

Part	Technology	Where it runs
Game client	Unity 6000.3.11f1, Universal Render Pipeline 17.3, C#, uGUI, Input System 1.19, Newtonsoft JSON 3.2, UnityWebRequest	Android phone (IL2CPP build); the Unity Editor while developing
Web API	Java 21, Spring Boot 4.1 (Web, Security, OAuth2 Resource Server, Data JPA, Validation, Actuator, Mail, Flyway)	Docker container on Railway, rebuilt on every push to GitHub <code>main</code>
Database	PostgreSQL; schema managed by 11 Flyway migrations	Railway
AI	Google Gemini (<code>gemini-2.5-flash</code> by default) through the server's <code>/api/ai</code> proxy	Google; key held on the server only
Soil data	ISRIC SoilGrids v2.0 REST API (sand, silt, clay, pH, organic carbon, coarse fragments, bulk density, nitrogen at 0–5 cm)	Called by the phone
Email	Resend HTTPS API on the cloud, Gmail SMTP locally	Called by the server

1.2 The player's journey through the three scenes

1. **MainMenu scene.** An intro video plays, then the menu. If no remembered login exists, the login board appears (`AuthUIBuilder`).
 - Sign-up and login are two steps: the password is checked, then a 6-digit code is emailed.
 - Pressing Start asks the server for the saved farm (`GET /api/farms/me`).
 - If a farm exists it is loaded into the farm scene. If the server answers 404 (no farm yet), the game goes to area selection.
2. **AreaSelection scene.** The player picks one of the Davao City districts, then drags a box over farmland inside it. Generate checks the box against the district's "agricultural area" picture and stores the choice in `SelectedAreaState` .
3. **TerrainPreview scene (the farm).** `SoilAwareTerrainGenerator` builds a 700 × 700 m terrain from the official Davao elevation and slope maps. It then gets two soil samples from SoilGrids and blends them across the farm. When a saved farm is being loaded, everything in it is restored.
 - A new farm gets the beginner tutorial with Antonio.
 - From then on the player farms, and saves to the server with the Save Farm button or the pause menu.

1.3 Design patterns used across the Unity code

Pattern	What it means here	Examples
UI built in code	No UI prefabs. Every panel is created at runtime by a <code>*UIBuilder</code> class. Art comes from one ScriptableObject (<code>UIThemeSprites</code> , the <code>Resources/UITheme</code> asset), and an empty art slot falls back to a plain colour.	<code>ShopUIBuilder</code> , <code>SeedlingTentUIBuilder</code> , <code>FarmMapUIBuilder</code>
Runtime bootstraps	Static methods marked <code>[RuntimeInitializeOnLoadMethod]</code> create the systems when a scene loads, so scenes need almost no manual wiring.	<code>BackendRuntimeBootstrap</code> , <code>FarmTaskRuntimeBootstrap</code> , <code>NurseryRuntimeBootstrap</code>
Singletons	One instance per system, reached through a static <code>Instance</code> property.	<code>PlayerInventory</code> , <code>GameTimeSystem</code> , <code>NurserySystem</code>
Events	Systems announce changes with C# events instead of polling one another.	<code>GameTimeSystem.OnNewGameDay</code> , <code>WeatherSystem.OnWeatherChanged</code> , <code>FarmTaskActionHub.ActionRecorded</code>

Pattern	What it means here	Examples
Observer + hub	An observer scans world state and infers what the player did. Every completed action then passes through one hub, which removes duplicates and forwards it to daily tasks, AI tasks and the tutorial.	FarmTaskActionObserver → FarmTaskActionHub
Data-driven catalogs	Game rules are kept in tables rather than scattered through code.	PlantingMaterialCatalog, TropicalCropCatalog, DailyTaskCatalog, CropMaintenanceCatalog, PestDiseaseDatabase
Partial classes	One large class is split across files by topic.	FarmingInteractionSystem (+ .Planting), SocialMarketplaceUIBuilder (.Core, .Marketplace, .PlayersChat, .Trade)
Snapshot persistence	The whole farm is captured into plain data classes (DTOs), serialised to JSON and stored by the server as one JSONB document, protected by a revision number.	FarmPersistenceManager, FarmSnapshotModels, CropPersistenceMapper
Adapter	Coconut, banana and the eleven other crops are three unrelated classes. An adapter lets other systems treat any of them the same way.	CropRuntimeAdapter, DevCropVitals

1.4 Game time and the simulation loop

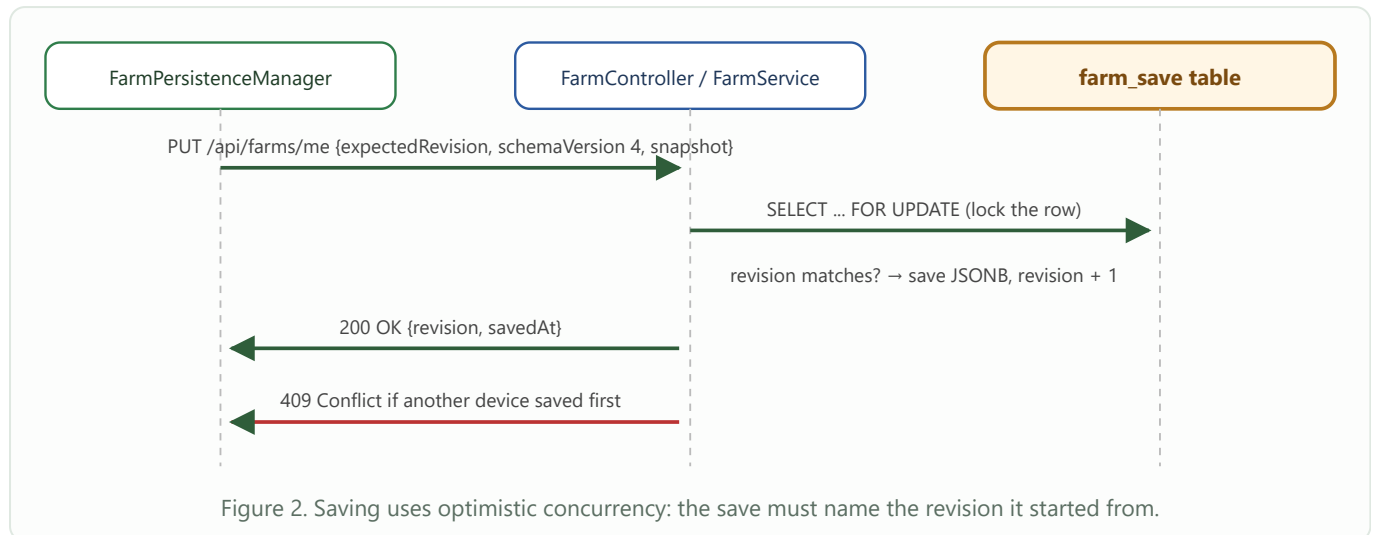
- **Clock.** One game day is 900 real seconds (15 minutes). A month has 16 days and a year has 12 months, so a game year is 192 days. A new farm starts at 08:00 (`GameTimeSystem`).
- **New day.** Each new day raises `OnNewGameDay`. The weather rolls the next day, pests and diseases roll their daily risk, daily tasks are drawn, and the Seedling Tent's timers keep counting.
- **Crops.** Each crop updates itself every frame. Its moisture dries by weather, its stress and health follow water, soil, temperature and weather, it moves through growth stages by age, and it produces fruit on a cycle.
- **Skipping time.** Skipping days (the objectives book or the developer tools) runs the same rules in bounded steps, so a skipped day has the same effect as a played one.
- **Weather.** Rolled daily from monthly profiles of Davao's climate: temperatures, humidity, rainfall, rainy days and sunshine, plus chances of rain, typhoon and extreme drought (`WeatherSystem`).
- **Grace period.** The first two game days on a new farm are always calm (`FarmGraceperiod`).

1.5 The main data flows

Signing in

1. POST `/api/auth/login/request` with an email or display name and the password. The server checks the password (BCrypt) and emails a 6-digit code.
2. POST `/api/auth/login/verify` with the code. The server returns a JWT (valid 30 days by default), the user and whether a farm exists.
3. The phone remembers the token (`SavedSession`), so a returning player lands on the menu.
4. Changing the password raises the account's **token version**, which ends every session issued before it.

Saving the farm



Marketplace and trades

The server moves items and money itself by editing the saved JSON of both farms. The phone saves first (`SaveBeforeEconomy`) so the server has its latest state. The server then:

- locks both farm rows in a fixed order, so two requests cannot deadlock;
- checks money, stock, inventory space and district rules;
- changes both snapshots in one database transaction.

Back on the phone, `EconomyRevisionMerge` performs a three-way merge. It applies the server's change to the live inventory without losing anything the player did locally in the meantime.

One phone per account

While in the farm, the game sends a presence heartbeat every 15 seconds (`POST /api/presence/heartbeat` with a per-install device ID).

- If a different device is already active (seen within the last 45 seconds), the server answers 409 and the second phone is refused.
- If the first phone has gone quiet for longer than that, a new phone may take over the account. The old phone then ends its own session (`SessionTakeover`).

AI adviser

1. The game builds the prompt: the adviser's instructions, a JSON summary of the farm, and the player's question.
2. It sends them to `POST /api/ai/generate`.
3. The server puts its own rules (`AiGuardrails`) above the game's instructions, quotes the player's question so it is treated as data, and calls Gemini with the key held on the server.
4. If the AI is switched off or fails, Antonio answers in character that he is busy.

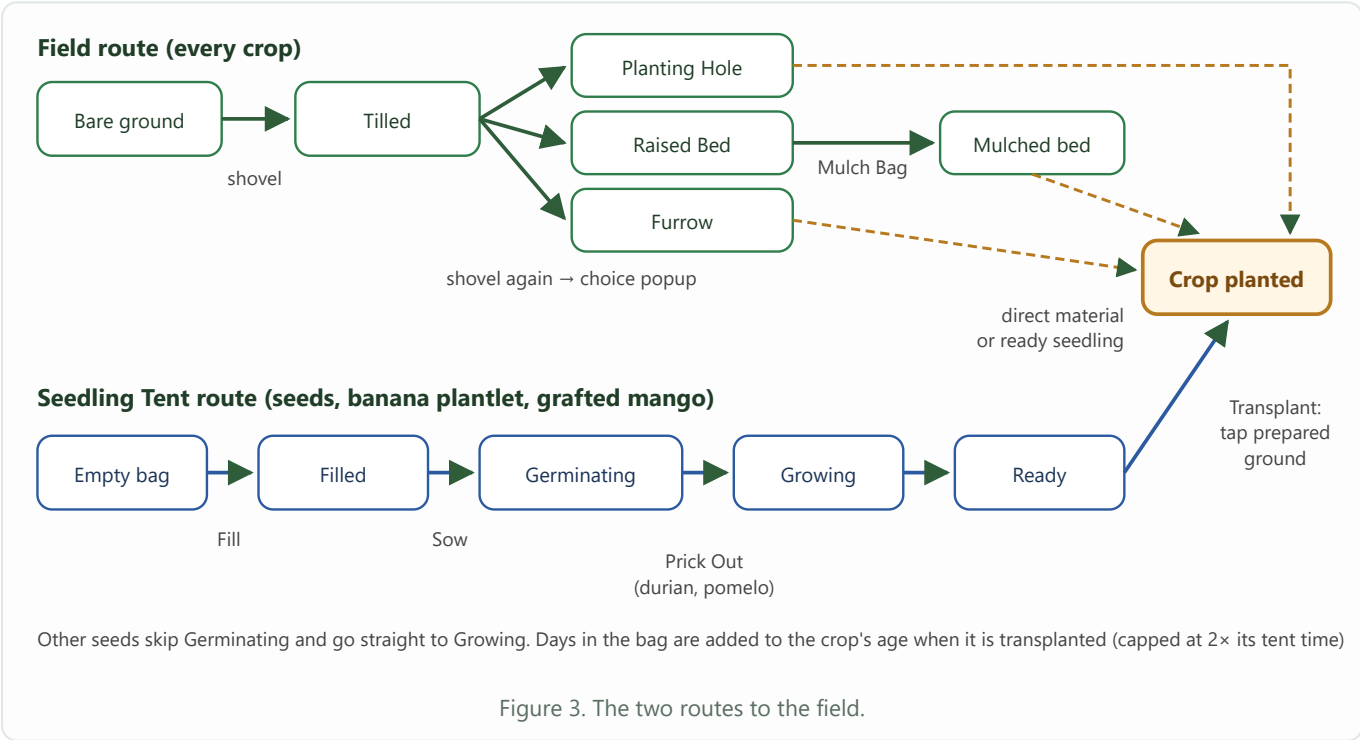
Part 2 — The planting system

The newest feature, explained end to end.

2.1 What the planting system changed

Before, a seed was tapped onto dug ground and a crop appeared. Now planting follows what Davao farmers actually do:

- **Ground preparation.** The shovel tills bare ground. Tapping the tilled patch again opens a choice: Planting Hole (tree crops, banana, coconut), Raised Bed (tomato, eggplant, squash, pineapple, strawberry) or Furrow (corn). A crop only goes into its own kind of ground. A raised bed physically lifts the terrain, and a Mulch Bag can cover it. The shovel can fill a plot back in.
- **Real planting materials.** The five old generic seeds became the materials farmers use: Banana Sucker, Coconut Seednut, Mango Liso, Pineapple Sucker and Strawberry Runner. Three were added: Banana Plantlet, Grafted Mango Seedling and Squash Seedling. There are 16 materials in all.
- **Seedling Tent (nursery).** A tent with 8 bags stands on every farm. Seeds, plantlets and grafted seedlings are raised there, then transplanted. Durian and pomelo must be pricked out of the seed tray by hand. The days in the tent count toward the crop's age.
- **Knock-on effects.** A crop planted on a raised bed or mulched bed inherits the drainage or mulch. Young cacao needs shade. The crop board shows "Planted From" and "In real farms". For the first 3 game days the crop is drawn as the real material.



2.2 Per-crop rules

Crop	Planting material	How it reaches the field	Ground	Special rule
Cacao	Cacao Seed	Tent, 6 days	Planting Hole	Young cacao needs shade. Without a Shade Net or greenhouse over it, it gains +6 stress per day until its pre-fruiting stage.

Crop	Planting material	How it reaches the field	Ground	Special rule
Durian	Durian Seed	Tent. Germinates in a seed tray, needs Prick Out after 2 days, ready 4 days later	Planting Hole	–
Mangosteen	Mangosteen Seed	Tent, 8 days (the longest)	Planting Hole	–
Pomelo	Pomelo Seed	Tent. Prick Out after 2 days, ready 4 days later	Planting Hole	–
Banana	Banana Plantlet <i>or</i> Banana Sucker	Plantlet: hardens 4 days in the tent. Sucker: planted directly	Planting Hole	Plantlet ₱30, sucker ₱15
Mango	Grafted Mango Seedling <i>or</i> Mango Liso	Grafted: hardens 2 days in the tent. Liso: planted directly	Planting Hole	Grafted starts 35 game days older, so it fruits sooner (₱80 vs ₱30)
Coconut	Coconut Seednut	Planted directly (sold already sprouted)	Planting Hole	–
Pineapple	Pineapple Sucker	Planted directly	Raised Bed	–
Strawberry	Strawberry Runner	Planted directly	Raised Bed, mulched first	Refused on a bed without mulch
Tomato	Tomato Seed	Tent, 3 days	Raised Bed	–
Eggplant	Eggplant Seed	Tent, 4 days	Raised Bed	–
Squash	Squash Seed <i>or</i> Squash Seedling	Seed: planted directly, or 2 days in the tent. Seedling: bought ready, already 2 days old	Raised Bed	The only crop with three routes (₱3,000 vs ₱3,300)
Corn	Corn Seed	Planted directly	Furrow	–

Tent times are game days, and a game day is 15 real minutes. The real nursery times would take many hours of play (cacao spends 5–12 months in a real nursery). The game compresses them and shows the real figures on the crop board's "In real farms" line. All real-world figures come from the project's own notes: `Handoff/crop_materials.txt` and `crop_production_notes.txt`.

2.3 How it works technically

The material decides the route, not the crop

`PlantingMaterialCatalog` holds one `PlantingMaterialInfo` per material with these fields:

- `Crop`: the crop it grows into.
- `SowInBag`, `PlantDirect` and `ReadySeedling`: which routes to the field it may take.
- `Plot`: the kind of ground it needs.
- `NeedsMulchedBed`: whether its raised bed must be mulched first.
- `NurseryDays`, `PrickAfterDays`, `ExtraAgeDays` and `ReadySeedlingAgeDays`: its timings.
- `RealWorld`: the real-farm description text.

Every other system asks the catalog instead of hard-coding crops.

Starting age when planted

```
ready seedling: age = ReadySeedlingAgeDays + ExtraAgeDays
from the tent: age = min(days in bag, 2 × NurseryDays) + ExtraAgeDays
planted directly: age = 0
```

`ApplyStartingAge` on the crop moves its planted day back by that many days and recomputes its growth stage and next harvest day. That is how tent days "count toward the crop's age".

How a tap is routed

`FarmingInteractionSystem.HandleInteractAtScreenPosition` casts a ray from the tap and calls `TryHandlePlantingTap` first:

1. Tapping the tent opens the tent panel.
2. While a seedling is being carried, tapping prepared ground transplants it. The one exception is the shovel on tilled ground, which still opens the choice popup.
3. Otherwise, tapping prepared ground is handled by the item in hand:
 - shovel: choose the plot kind, or fill the plot in;
 - Mulch Bag: mulch a raised bed;
 - planting material: plant it;
 - anything else: describe the ground.
4. The shovel on bare terrain tills (`TryTillGround`). It is refused within 8 m of the tent or too close to a crop or another plot.

Planting safely

`PlantOnPreparedGround` works in this order:

1. reserve the plot;
2. create the crop;
3. check spacing;
4. take the material from the inventory, only now;
5. apply the starting age;
6. let the crop adopt the plot's raised bed or mulch (`CropClimateMaintenanceState.AdoptPreparedGround` : +0.18 drainage for a bed; +0.08 moisture and -4 stress for mulch);
7. consume the plot;
8. show the crop's board.

If any step fails, the crop is removed and nothing is lost or duplicated.

Raised beds change the terrain

`TerrainModificationService.ApplyRaisedBed` lifts a mound of heightmap vertices (radius 3.2 m, 0.6 m high) and records a patch ID so a reload never applies it twice. `RemoveRaisedBed` applies the exact inverse when a bed is filled in.

Saving (schema version 4)

The farm snapshot gained three things:

- `nursery` : the tent's position and each bag's material, sow day and prick day;
- `preparedPlots` : unplanted plots, with kind, mulch, bed patch, soil and position;
- new crop fields: `plantingMaterial` , `nurseryDays` , `fieldPlantedGameDay` , `hasFruitBag` and `hasDrainageImprovement` .

Older saves still load. Missing fields keep their defaults, a tent is placed, and old seed names are upgraded (`PlantingMaterialCatalog.UpgradeLegacy`) in inventories, starting seeds, tasks and marketplace items. The server stores the snapshot as JSONB and keeps fields it does not know, so no database migration was needed.

Everything the change touched

System	What changed
Tutorial	Step 3 teaches till → prepare → plant and moves on only for the plot kind it asked for, with a hint that explains how to recover. New Step 4B introduces the Seedling Tent.
Daily tasks	New tasks 201–204: sow a bag, sow two bags, transplant, till new ground. "Plant" tasks are offered only when something can reach the ground today (<code>PlantingAvailability</code>).
AI adviser and AI tasks	The farm context lists the tent's bags and unplanted ground. The prompts carry the planting rules, and the checks know which material belongs to which crop.
Shop and descriptions	Sells all 16 materials. Each description has HOW TO PLANT and IN REAL FARMS sections.
Marketplace and trades	New materials are tradable on both sides with base values. The server's district rule (<code>DistrictSeedPools</code>) now works per crop.
Climate systems	A raised bed or mulched bed carries over to the crop. A shade net or greenhouse shades young cacao. Structures cannot be built next to the tent.
Map, developer tools, audio	A tent marker on the map. Material buttons and "Seedlings Ready" in the developer tools. Sound hooks for tilling, transplanting, filling a bag and watering.
Assets	Meshy 3D models in <code>Assets/Models</code> , prefabs in <code>Assets/Prefab</code> and UI images in <code>Assets/Sprites/UI</code> , all built and wired by <code>Editor/PlantingAssetsSetup.cs</code> .

Part 3 — Unity game: every script

202 C# files: 193 game scripts under Assets/Scripts and 9 editor tools under Assets/Editor, grouped by folder.

Every file is listed, each with what it does and the parts to point at when explaining it. Groups of near-identical files (for example the eleven small crop classes) share one entry.

3.1 Area selection: district data

Assets/Scripts/AreaSelection/Districts

The facts about Davao City's districts that the area selection screen reads.

DavaoGeoReference.cs

Converts a point on the area-selection map into real latitude and longitude. It uses the four corner coordinates of the Davao satellite map (about 7.61°N to 6.94°N and 125.14°E to 125.72°E). It also picks the two soil sample points, a quarter and three quarters of the way across the player's box, that are sent to SoilGrids.

Key parts: `LatLonFromNormalized`, `GetSamplePointA`, `GetSamplePointB`

DistrictAreaMap.cs

Holds the three pictures each district needs: the district highlight, the barangay highlight, and the "agricultural area spot" map. It reads the last one to decide whether the player's box is on farmland (green means allowed, red means not). Pixels are classified by which colour channel leads, so slight colour differences still read correctly, and decoded pictures are cached.

Key parts: `DistrictMapArt`, `DistrictAreaSpot.Validate`, `TryGetFootprint`, `AreaSelectionVerdict`

DistrictDescriptions.cs CHANGED · planting

The text behind "Check Description" for each district: an introduction, the terrain, and the STARTER CROPS section. The starter crops are read from `DistrictCropPools`, so the text always matches what the game actually gives. Updated to describe planting materials instead of seeds.

Key parts: `For(districtName)`

3.2 Area selection: the screen

Assets/Scripts/AreaSelection/Selection

Builds and runs the screen where a new player chooses where the farm will be.

AreaSelectionBuilder.cs CHANGED · planting

Builds the whole area selection screen in code, in two steps. In step 1 the player chooses a district with arrows and the camera glides to it. In step 2 they drag a box over its barangays, and Generate validates the box and loads the farm scene. It also handles the background video, the district description board and the optional tutorial slides. Updated wording: "...and the crops you can plant there".

Key parts: `Build`, `ApplyPhase`, `SwitchToDistrict`, `OnGeneratePressed`, `RefreshMapOverlay`

DraggableSelectionBox.cs

The box the player drags on the map. It stays inside the visible map area and reports its rectangle in 0–1 map coordinates.

Key parts: `GetNormalizedRect`, `OnDrag`

LoopingVideoBackground.cs

Plays a looping background video reliably on Android. A watchdog restarts it at loop points, after errors, after the app returns from the background or the keyboard closes, and whenever the frame counter stops moving. It also rebuilds the render texture if it was lost.

Key parts: `Create`, `IsStalled`, `RecreateRenderTexture`

MapZoomPanController.cs

Pinch and scroll-wheel zoom and drag panning of the map, kept inside the view, with a smooth "glide" to focus on a district.

Key parts: `ZoomAtScreenPoint`, `GlideToNormalizedPoint`, `ClampContentToViewport`

SelectedAreaState.cs

A static holder for the player's choice, the rectangle and the district name, carried from this scene into the farm scene.

Key parts: `NormalizedRect`, `SelectedDistrictName`

TutorialSlideBoard.cs

A picture-book tutorial shown one page at a time, turned with the >> plank. It covers the screen behind it so the explained buttons cannot be pressed while a page is open.

Key parts: `TutorialSlide`, `Show`, `Hide`

3.3 Main menu and accounts

Assets/Scripts/MainMenu

The first scene: the intro, the menu, and every account screen.

Account/AccountRules.cs

Client-side copies of the server's account rules, such as the email format, password length and display name length. The player sees a problem the moment they press the button. The server remains the authority, and these rules are never stricter than the server's.

Key parts: `SignupProblem`, `ResetPasswordProblem`, `ChangeEmailProblem`, `ChangePasswordProblem`, `ChangeDisplayNameProblem`

Account/AccountUIBuilder.cs

The Account board opened from the main menu. It shows the display name and email and offers three changes: email (confirmed with a code sent to the new address), password, and display name.

Key parts: `ShowDetails`, `OnSubmitNewEmail`, `OnVerifyEmailChange`, `OnSubmitPassword`, `OnSubmitDisplayName`

Account/AuthUIBuilder.cs

The login, sign-up, verify-code, forgot-password and reset-password panels. Each button calls the matching `AuthSession` method.

Key parts: `ShowLogin`, `ShowSignup`, `OnLoginPressed`, `OnVerifyPressed`, `OnForgotSubmitPressed`, `OnResetSubmitPressed`

Menu/CreditsUIBuilder.cs

Rolling credits. They scroll by themselves, pause while dragged, resume shortly after release, and stop at the end.

Key parts: `Show`, `Hide`, `CreditsDragWatcher`

Menu/MainMenuBuilder.cs

The main menu. It plays the intro video and runs the looping background video with a stall watchdog, animates the title, and shows the Start, Settings and Exit buttons plus Account, Credits and Logout. If nobody is signed in it shows the login board. Start hands over to `AuthSession.StartGame`, which opens the saved farm or, for a new player, area selection.

Key parts: `Build`, `OnStartPressed`, `ShowAuthenticationIfNeeded`, `OnLogoutConfirmed`

3.4 Shared services (used in every scene)

Assets/Scripts/Shared

Audio, the backend connection, district rules, settings and the reusable UI helpers.

Accessibility/GameAccessibility.cs

Makes the game readable by Android accessibility services such as TalkBack. Unity draws everything into one Android view, so this component labels that view and publishes a list of every on-screen control, rebuilt when the screen changes. It never moves or restyles any UI.

Key parts: `Rebuild`, `Collect`, `LabelForControl`

Audio/GameAudioManager.cs CHANGED · planting

The central audio engine. It plays background music and weather ambience with crossfades, plus one-shot sound effects (click, dig, plant, spray, reward), and attaches click sounds to new buttons automatically. It survives scene loads. New for planting: `PlayTill`, `PlayTransplant` and `PlayFillBag`, which reuse the dig and plant sounds until their own clips are assigned, and `PlayWater`, which stays silent until a clip is assigned.

Key parts: `PlaySfx`, `PlayMusic`, `PlayAmbience`, `Crossfade`

Audio/SceneMusicPlayer.cs

A small per-scene component that tells `GameAudioManager` which track to play whenever its scene loads.

Key parts: `music`

Backend/ApiClient.cs

The HTTP layer. It sends JSON requests to the backend with `UnityWebRequest` (GET, POST, PUT), adds the `Authorization: Bearer` token, and converts JSON with `Newtonsoft`. Server errors become readable messages plus the HTTP status code. It is coroutine-based and reports through success and error callbacks.

Key parts: `GetJson`, `PostJson`, `PutJson`, `PostWithoutBody`, `ExtractError`

Backend/ApiConfiguration.cs

Chooses the backend address. Phones, and the Editor by default, use the public Railway HTTPS address. Otherwise it falls back to localhost, the Android emulator or a LAN address for local testing. It warns when a plain-HTTP address would be blocked by Android.

Key parts: `BaseUrl`, `IsCleartext`

Backend/AuthSession.cs

The signed-in session. It holds the access token, the current user and the loaded farm, and runs every account flow: register, login with an emailed code, password reset, and email, password and display name changes. It restores a remembered login at launch and claims the play session for this device. It loads the farm (`GET /api/farms/me`) and opens the farm or, on 404, area selection. It also syncs settings and logs out.

Key parts: `RequestLogin/VerifyLogin`, `StartGame`, `LoadFarmAndOpenWorld`, `ApplySavedFarm`, `ClaimPlaySession`, `Logout`, event `SessionChanged`

Backend/BackendModels.cs

The data-transfer objects (DTOs) that mirror the server's JSON: auth requests and responses, the user, admin tools, the presence heartbeat, player settings, AI generate requests and responses, and the farm save request and response.

Key parts: `AuthResponseDto`, `FarmSaveRequestDto`, `AIGenerateRequestDto`, `SettingsDto`

Backend/BackendRuntimeBootstrap.cs

Runs before the first scene and creates the persistent backend objects (`ApiConfiguration`, `ApiClient`, `AuthSession`). It adds the login UI in `MainMenu`, and the persistence, Save button and pause menu in the farm scene. This is why scenes need no manual wiring.

Key parts: `CreateBootstrap`, `OnSceneLoaded`

Backend/DeviceIdentity.cs

A random ID created once per installation and kept in `PlayerPrefs`. It tells devices apart for the one-phone-per-account rule and reveals nothing about the phone or its owner.

Key parts: device ID property

Backend/SavedSession.cs

Remembers the login between launches (the token and the email) so returning players skip the login board. Logout erases it. Passwords are never stored.

Key parts: `Save`, `TryLoad`, `Clear`

Backend/SessionTakeover.cs

Ends this device's session when another phone takes the account over, which the heartbeat learns from an HTTP 409. It deliberately does not save on the way out, because the other device now owns the farm. The player is then shown why.

Key parts: `Raise`, `RaiseSignedOut`, `Consume`

Districts/DavaoDistrictService.cs

Reads the coloured district border map to find which district a map point is in. It validates that a selection lies inside a farming district and computes each district's bounds so the camera can frame it.

Key parts: `TryGetDistrictAtNormalized`, `ValidateSelection`, `TryGetDistrictNormalizedBounds`

Districts/DistrictCropPools.cs CHANGED · planting

Which crops each district grows (Calinan, Toril, Baguio, Paquibato, Marilog, Buhangin, Tugbok, plus a fallback), and so which planting materials a farm may obtain. It rolls the three starting materials for a new farm, always including one that can be planted directly. It must match the server's `DistrictSeedPools`. Rewritten around crops, so every material of an allowed crop is allowed and old seed names are recognised.

Key parts: `CropsFor`, `PoolFor`, `IsAvailableIn`, `RollStartingSeeds`, `Describe`

Settings/GameSettings.cs

The store for player settings: three volumes, render distance, interface size, text size and AI summarisation. It is cached in `PlayerPrefs`, replaced by the account's copy from the server after login, and raises `Changed` so other systems react immediately.

Key parts: `Set*`, `ApplyFromDto`, `ToDto`, event `Changed`

Settings/MobileOrientationLock.cs

Locks the screen to landscape on phones.

Key parts: `Awake`

Settings/MobilePerformanceBootstrap.cs

Applies the phone performance settings, the frame-rate cap and shadows, in code before the first scene. Every device then gets the same tuning whatever quality tier it lands on.

Key parts: Apply

Settings/PauseMenuBuilder.cs

The in-farm pause menu. It shows the player's name and district, Settings, and Main Menu. Main Menu asks whether to save first: Yes saves then leaves, No leaves, Cancel stays. The farm keeps simulating while the menu is open.

Key parts: OpenPauseMenu, SaveThenLeave, ApplySpriteTint (shared helper)

Settings/SettingsUIBuilder.cs

The Settings overlay: six sliders (music, sound effects, ambience, render distance, interface size, text size) and the AI summarisation toggle. Changes apply live. Closing asks whether to save, to the phone and to the account.

Key parts: Show, OnSaveYes, RestoreBaseline

Settings/UIScaleService.cs

Applies the interface and text size settings to every canvas and every label in every scene. Because all UI is built at runtime, it re-scans on a short interval instead of hooking each builder. It also rescues text that would otherwise be cut off.

Key parts: ApplyCanvasScale, ApplyTextScale, RescueFromTruncation

UI/DescriptionBoard.cs

A reusable wooden board for long scrollable descriptions, used for shop items, districts and the tent's materials, with a CLOSE plank.

Key parts: Create, CreateOpenButton, Show, Hide

UI/GameFonts.cs

The one place that chooses the game's font. It is loaded from Resources, and Unity's built-in font is the fallback.

Key parts: Primary

UI/HoldButton.cs

A button that fires as soon as it is pressed down rather than on release.

Key parts: onPressed, OnPointerDown

UI/HudIconButton.cs CHANGED · planting

Creates the round wooden HUD buttons along the top edge and places them by slot number, so buttons made by different scripts never overlap. Slot 7 was added for the Seedling Tent.

Key parts: Create, PositionForSlot, SlotSeedlingTent

UI/HudRegistry.cs CHANGED · planting

A registry of HUD pieces (hotbar, joystick, panels and so on). The tutorial uses it to reveal them one at a time, and opening one panel closes the others. The Seedling Tent panel was added.

Key parts: RegisterPiece, SetPieceVisible, BeginTutorialControl, CloseOtherPanels

UI/MobileTextEntryOverlay.cs

Shows what is being typed in a bar at the top of the screen, because the phone keyboard covers text boxes in landscape. It watches the EventSystem, so every text box in the game is covered automatically.

Key parts: ResolveFocusedField, BuildDisplayText

UI/UIPlank.cs

Builds wooden planks with live text on them, for headings and buttons, from blank plank art.

Key parts: Create, CreateButton, SizeFor

UI/UIThemeSprites.cs CHANGED · planting

The ScriptableObject (the Resources/UITheme asset) that holds every UI sprite in one Inspector, so runtime-built panels can find their art. An empty slot falls back to a plain colour. Slots were added for the tent button, the bag pictures and the bag icons.

Key parts: UIThemeSprites.Instance, GetDistrictSign

3.5 AI farm adviser (Antonio chat)

Assets/Scripts/TerrainPreview/AIAdviser

The chat with Antonio. The game builds the prompt, and the server calls Gemini.

AiBackendClient.cs

Sends every AI request to the game's own server (POST /api/ai/generate) with the player's JWT instead of calling Google directly, so the Gemini key never ships inside the APK.

Key parts: Generate

AiResponseStyle.cs

Adds a "keep it brief" instruction to the adviser and climate-evaluation prompts when the player has turned on AI Summarization. The two task features are left alone because they must return exact JSON.

Key parts: Apply

AiText.cs

Strips Markdown (****bold****, **##** headings, ***** bullets) from Gemini replies so text boxes show clean text. Underscores are kept, because crop names like mangosteen_1 contain them.

Key parts: StripMarkdown

FarmAdvisorChatSystem.cs

Connects the chat UI, the context builder and the Gemini client. When Send is pressed it builds the farm context, sends the question, and shows Antonio's answer.

Key parts: OnSendPressed, SetBusy

FarmAdvisorContextBuilder.cs CHANGED · planting

Collects the farm's current state as JSON for the adviser: date, weather, soil, crop groups (average health, stress, water, drainage, fertility and suitability) and pests. The Seedling Tent's bags were added.

Key parts: BuildContext, BuildContextJson, BuildSeedlingTent

FarmAdvisorGeminiClient.cs

Holds the adviser's prompt, editable in the Inspector, and sends the question through AiBackendClient.

Key parts: AskAdvisor

FarmAdvisorToolReference.cs CHANGED · planting

The list of things the player can actually do in the game, such as each item and how effective it is in each weather. It is appended to the adviser's instructions so Antonio recommends real in-game actions. A planting section was added: materials, tent and plots.

Key parts: reference text

FarmAdvisorUIBuilder.cs

The chat panel: Antonio's animated portrait (three looping videos), the message area, the input box and the Send button.

Key parts: ToggleChat , AppendMessage , SetMood

3.6 Farm audio

Assets/Scripts/TerrainPreview/Audio

FarmAudioController.cs

Farm-scene audio. It crossfades day and night music and switches the weather ambience (clear, rain, typhoon, drought) when the weather changes.

Key parts: ApplyDayNight , HandleWeatherChanged

3.7 Climate event evaluation

Assets/Scripts/TerrainPreview/ClimateEvaluation

Grades how well the player handled a typhoon or extreme drought.

ClimateEvaluationModels.cs

Data classes for the evaluation: each recorded action (type, item, crop, before and after moisture, health, stress, drainage and fertility, effectiveness score), crop snapshots, and the payload sent for grading.

Key parts: ClimateActionRecord , CropSnapshot , ClimateEventEvaluationPayload

ClimateEvaluationPopupBuilder.cs

The popups shown when a climate event starts and when it ends. The ending popup shows the score and Antonio's written evaluation.

Key parts: ShowEventStarted , ShowEventEnded

ClimateEventTracker.cs

Follows a climate event from start to finish. It snapshots every crop at the start and records every action taken during the event, removing duplicates. At the end it snapshots again and computes a local 0–100 score: a base of 45, plus each action's effectiveness, plus the average change in health, stress and moisture, minus 15 if nothing was done. It then asks Gemini for a written evaluation. It is also the route by which actions reach the task hub.

Key parts: RecordAction , RecordObservedAction , StartTracking , FinishTracking , ComputeScore

ClimateResilienceGeminiClient.cs

Holds the evaluation prompt and sends it through the server as the CLIMATE_EVALUATION feature.

Key parts: EvaluateClimateEvent

3.8 Crop care and climate structures

Assets/Scripts/TerrainPreview/ClimateMaintenance

Crop care items (mulch, stakes, trellis, raised bed kit, compost, pruning) and placed structures (irrigation, water tank, shade net, windbreak, greenhouse, drainage canal).

ClimateActionAutoObserver.cs CHANGED · planting

The older observer that recorded actions during climate events by watching the world.

`FarmTaskActionObserver` switches it off at runtime to avoid duplicate records. It was kept compatible, recording planting material names.

Key parts: `Scan`, `ScanCropLifecycle`

ClimateMaintenanceInteractionSystem.cs CHANGED · planting

Handles taps with climate items. It applies crop care to a crop and places structures on the terrain, with reach and spacing checks. Updated so taps on the Seedling Tent and on prepared ground are left to the planting system, and structures cannot be built next to the tent.

Key parts: `HandleInteractAtScreenPosition`, `HandleCropAction`, `HandleWorldPlacement`

ClimateMaintenanceRuntimeBootstrap.cs

Creates the climate-maintenance systems in the farm scene at runtime.

Key parts: `EnsureInstalled`

ClimateMaintenanceToastUI.cs

A short temporary message ("toast") for climate actions.

Key parts: `Show`

ClimateMaintenanceTypes.cs

Enums for crop-care actions and structure types, plus the item catalog that maps inventory items to them (names, prefab names, reusable tools).

Key parts: `CropMaintenanceActionType`, `ClimateWorldMitigationType`, `ClimateMaintenanceItemCatalog`

ClimateMitigationWorldObject.cs CHANGED · planting

The base class for placed structures. Each structure has a radius and an effectiveness, and affects nearby crops every frame and during time skips. For example, irrigation adds water drawn from the nearest tank, and a canal reshapes the terrain. It saves and restores itself and shows inspection text. New: a registry of active structures and `IsShadeOver`, which tells whether a shade net or greenhouse covers a young cacao. Inspection text now uses separate lines (a bug fix).

Key parts: `IsShadeOver`, `SimulateTimeSkip`, `CaptureSaveData`, `RestoreFromSave`, `GetInspectionText`

CropClimateMaintenanceState.cs CHANGED · planting

Per-crop care flags (mulch, support stake, trellis, raised bed, last compost and prune day), their effects and visuals, and saving. New: `AdoptPreparedGround`, so a crop planted on a raised bed or mulched bed inherits it (+0.18 drainage for a bed; +0.08 moisture and -4 stress for mulch).

Key parts: `ApplyAction`, `AdoptPreparedGround`, `SimulateTimeSkip`

CropMaintenanceCatalog.cs

The rules for which care actions each crop allows and how effective each action or structure is in each weather event. Some are harmful, such as irrigation during a typhoon or a drainage canal in a drought.

Key parts: `IsAllowed`, `ScoreCropAction`, `ScoreWorldAction`, `MaintenanceGuide`

CropRuntimeAdapter.cs CHANGED · planting

A wrapper that lets other systems treat the three crop classes the same way: read or change health, stress, moisture, fertility and drainage, and find every crop. It now also exposes the planting material and the nursery days.

Key parts: TryCreate, FindAll, Add*, PlantingMaterial, NurseryDays

DrainageCanalInstance.cs · GreenhouseInstance.cs · IrrigationSystemInstance.cs · ShadeNetInstance.cs · WaterStorageTankInstance.cs · WindbreakInstance.cs

Six small subclasses of `ClimateMitigationWorldObject`. Each sets its structure type and default radius and effectiveness, and draws its radius in the Editor.

Key parts: Reset, OnValidate, OnDrawGizmosSelected

TerrainModificationService.cs CHANGED · planting

Edits the terrain heightmap. It raises a mound for a raised bed and cuts a channel for a drainage canal, remembering which patches it applied so a reload never applies them twice. New: `RemoveRaisedBed`, the exact inverse used when a bed is filled in, with a shared `ApplyMound`.

Key parts: SnapToHeightmapVertex, ApplyRaisedBed, RemoveRaisedBed, ApplyDrainageCanal

3.9 Crop simulation

Assets/Scripts/TerrainPreview/CropInstances

One component per planted crop that simulates it over time.

BananaPlantInstance.cs CHANGED · planting

Simulates one banana plant: growth stages, water, stress, health, soil suitability, temperature and weather effects, bunch production and harvest, and its information text. New: the planting material, nursery days and field-planted day, `ApplyStartingAge`, and the "Planted From" and "In real farms" lines.

Key parts: Initialize, SimulateTimeSkip, Water, HarvestAllBulbs, ApplyStartingAge, GetInspectionText

CoconutTreeInstance.cs CHANGED · planting

The same for coconut trees: stages, production cycles and harvest. Updated in the same way as banana.

Key parts: Initialize, Harvest, ApplyStartingAge

TropicalCropPlantInstance.cs CHANGED · planting

The shared simulation for the other eleven crops (cacao, durian, mangosteen, pomelo, mango, pineapple, strawberry, tomato, eggplant, squash, corn), driven by each crop's numbers in `TropicalCropCatalog`. It supports both single-fruit and bundle harvests. New: the planting fields, `ApplyStartingAge`, and the young-cacao shade rule (+6 stress per day without shade).

Key parts: Simulate, HarvestIndividualFruits, HarvestBundles, NeedsShade, IsShaded

CacaoPlantInstance.cs · CornPlantInstance.cs · DurianPlantInstance.cs · EggplantPlantInstance.cs · MangoPlantInstance.cs · MangosteenPlantInstance.cs · PineapplePlantInstance.cs · PomeloPlantInstance.cs · SquashPlantInstance.cs · StrawberryPlantInstance.cs · TomatoPlantInstance.cs

Eleven tiny subclasses of `TropicalCropPlantInstance`. Each only sets which crop it is, so every crop prefab has its own component. (The strawberry file name has a triple "r".)

Key parts: Awake, Reset

TropicalCropTypes.cs CHANGED · planting

Enums for crop kind, growth stage and harvest mode, and `TropicalCropCatalog`. The catalog holds every crop's numbers: first-harvest delay, production interval, stage ages, ideal moisture and drainage, drying rate, typhoon and drought stress, soil and pH preferences, and yield formulas. `GetSeedItem` now returns the crop's default planting material.

Key parts: `GetFirstHarvestDelayDays`, `GetStageForAgeDays`, `CalculateSoilSuitability`, `CalculateIndividualYield`

3.10 Developer and test tools

`Assets/Scripts/TerrainPreview/DevTools`

Tools for testing and measuring. Admin-only tools are allowed or refused by the server.

DevCropVitals.cs

Reads and edits one crop's live readings for the developer tools, handling all three crop classes in one place.

Key parts: `TryRead`, `TryApply`

DevToolsUIBuilder.cs CHANGED · planting

The developer panel. It shows the weather and day, forces weather or pest events, gives money and items, passes days or hours, fills tanks and edits crop readings. For another player it can send events and delete a test account (server-checked admin accounts only). Buttons were added for all 16 planting materials and "Seedlings Ready".

Key parts: `ToggleDevPanel`, `OnSendEventPressed`, `OnDeleteAccountConfirmed`

FpsCounterHud.cs

The frame-rate readout for the performance requirement NFR-01, an average of at least 30 FPS over 10 minutes. It shows the live rate, the average, the lowest one-second rate and the elapsed time. Tapping it restarts the measurement.

Key parts: `Measure`, `StartMeasuring`

UiResponseTimerHud.cs CHANGED · planting

Measures how long each screen takes to appear after the tap, for NFR-02: visible feedback within 2 seconds. The Seedling Tent panel was added.

Key parts: `RegisterScreens`, `Record`

3.11 Shop, prices and selling

`Assets/Scripts/TerrainPreview/Economy`

PesoPrice.cs

Prices are held exactly in centavos, because the Davao City government price list has centavos, while wallets use whole pesos. A total is rounded once, with halves rounding up, using the same rule as the server's `PesoRounding`, so the fee shown equals the fee charged.

Key parts: `Centavos`, `TotalPesos`, `Label`

ShippingBinSeller.cs

A marker on the shipping bin, the place where harvested crops are sold.

Key parts: (marker component)

ShippingBinSpawner.cs

Places the shipping bin near the player on a new farm, or restores it from the save.

Key parts: SpawnBinNearPlayerNow, RestoreBin

ShopItemDescriptions.cs CHANGED · planting

The shop's description for every item, built from the simulation's own numbers: growth, yield, pests, care rules and temperature. Each planting material now has HOW TO PLANT and IN REAL FARMS sections.

Key parts: For(item), MaterialDescription, CropText

ShopUIBuilder.cs CHANGED · planting

The shop panel: shelf pages, item preview, price, and buying a quantity. Items the player's district does not grow are greyed out. It now stocks the 16 planting materials with their prices.

Key parts: StockedPriceCentavos, SellsToPlayer, TryBuySeed, RefreshShelf

3.12 Farming interactions

Assets/Scripts/TerrainPreview/Farming

Turns a tap on the farm into an action. The planting half is in its own file.

CropClimateRules.cs

The FarmCropType enum (the 13 crops) and each crop's comfortable temperature range, turned into a suitability score and a status.

Key parts: GetProfile, GetTemperatureSuitability

CropFieldDescriptions.cs CHANGED · planting

Explains every line on the crop information board, for example health versus average health, stress, water and drainage. Entries were added for PLANTED FROM, SHADE and IN REAL FARMS.

Key parts: description text

CropNaming.cs

Gives crops friendly names such as corn_1 and cacao_2, used on screen and by the AI. Internal IDs stay GUIDs.

Key parts: NextName, Humanize

CropProtectionState.cs CHANGED · planting

Per-crop protections that are not sprays, fruit bags and drainage kits, which reduce pest and disease pressure every day. A bug fix makes them survive a reload without doubling the drainage bonus.

Key parts: InstallFruitBag, InstallDrainageKit, RestoreSaveData

DigSpot.cs CHANGED · planting

A patch of prepared ground. It goes from tilled to a planting hole, a raised bed or a furrow, may be mulched, and keeps the soil sampled when it was tilled. It gained the PreparedPlotKind enum and the plot fields.

Key parts: PreparedPlotKind, IsPrepared, DisplayName, TryReserve, Consume

FarmingInteractionSystem.cs CHANGED · planting

The main farm interaction script. It turns a tap into an action based on the selected item: water, harvest, collect, sell to the bin, spray, place traps, fruit bags or drainage kits, or remove with the machete. It also creates and restores crops. Planting was moved to the `.Planting` partial file. It also restores planting details on load, plays the watering sound, and keeps the crop tap target at ground level.

Key parts: `HandleInteractAtScreenPosition`, `RestoreCropFromSave`, `TryWaterTropical`, `TryHarvestTropical`, `TrySellSelectedToBin`

FarmingInteractionSystem.Planting.cs NEW · planting

The planting half of the farming system. It covers tilling, the plot choice, raised beds and filling them in, mulching beds, planting the 16 materials, and transplanting from the tent. It also chooses each crop's just-planted model and saves and restores unplanted plots. See Part 2 for the full flow.

Key parts: `TryHandlePlantingTap`, `TryTillGround`, `ShowPrepareChoice`, `PrepareGround`, `TryMulchBed`, `TryPlantMaterial`, `TryTransplantInto`, `PlantOnPreparedGround`, `CapturePreparedGround`, `RestorePreparedGround`

GrowthStageVisualController.cs CHANGED · planting

Swaps a crop's 3D model as it grows (sprout, second stage, adult) and keeps it seated on the terrain. New: for the first 3 game days it shows the real planting material, and it keeps the tap target at ground level so small crops can be tapped (a bug fix).

Key parts: `ForceRefresh`, `GetTargetVisualStage`, `SeatPlantedVisual`, `KeepTapTargetOnGround`

3.13 Daily tasks and AI tasks

`Assets/Scripts/TerrainPreview/FarmTasks`

Daily objectives with rewards, plus Antonio's AI-generated tasks.

AIAdvisorTaskGeminiClient.cs CHANGED · planting

Asks the AI to invent a task and later to grade it, through the server with strict JSON output. The planting rules were added to its prompt.

Key parts: `GenerateTask`, `CheckTask`

AIAdvisorTaskSystem.cs CHANGED · planting

Antonio's task feature. It asks the AI for a multi-day task based on the farm, checks that the task can really be done, and tracks progress from recorded actions. It then grades the task with the AI (or locally) and pays the reward. Its checks now know about planting materials.

Key parts: `RequestTask`, `CheckTask`, `RecordAction`, `IsGeneratedTaskUsable`, `CanObtainSeedFor`

DailyTaskCatalog.cs CHANGED · planting

The 204 daily task templates: routine, general, drought, typhoon, pest and disease, advanced, and nursery. Added tasks 201–204 (sow a seedling bag, sow two bags, transplant a seedling, till new ground) and updated the templates for planting materials.

Key parts: `GetAll`, `AddNurseryTasks`

DailyTaskSystem.cs CHANGED · planting

Draws 2 tasks each game day, choosing only tasks that can actually be completed today. It tracks progress from recorded actions and sets the reward: ₱300, ₱400 or ₱500 by difficulty, or ₱500 when urgent. It allows a day skip and saves and restores its state. New task kinds, planting-availability checks and old-item upgrades were added.

Key parts: `GenerateDailyTasks`, `IsEligible`, `RecordAction`, `CollectReward`, `SkipToNextDayAtEight`

FarmTaskActionHub.cs

The single point every completed farm action passes through. It removes cross-source duplicates within 2 seconds, then forwards each action to daily tasks, AI tasks and the tutorial through the `ActionRecorded` event.

Key parts: `Record`, `ActionRecorded`

FarmTaskActionObserver.cs CHANGED · planting

Scans the world every 0.25 s and infers actions by comparing state. It detects crops planted or removed, watering, care added, conditions reduced, traps, structures, inventory and sales. New: it detects tilling, plot preparation and mulching a bed, and records the planting material on `PlantCrop`.

Key parts: `ScanCrops`, `ScanDigSpots`, `ScanInventoryAndSales`, `Route`

FarmTaskContextBuilder.cs CHANGED · planting

Builds the farm summary the AI tasks read: crops, conditions, inventory and structures. Tent bags and unplanted ground were added, along with crop-to-materials mapping.

Key parts: `Capture`, `CropNameToPlantingMaterials`

FarmTaskModels.cs CHANGED · planting

Data classes and enums for tasks: `DailyTaskKind`, templates, instances, the AI task objective and context payloads. The `TillGround`, `SowSeedlingBag` and `TransplantSeedling` kinds were added, with tent and prepared-ground contexts.

Key parts: `DailyTaskKind`, `DailyTaskInstance`, `FarmTaskContextPayload`

FarmTaskPopupUI.cs

A popup for task messages and results.

Key parts: `Show`, `Hide`

FarmTaskRuntimeBootstrap.cs

Creates the task systems (daily tasks, AI tasks, action observer, UI) at runtime and switches off the old climate observer.

Key parts: `EnsureInstalled`

FarmTaskUIBuilder.cs

The Objectives book: the daily tasks with their progress, the reward and skip-day buttons, and the AI task page.

Key parts: `TogglePanel`, `Refresh`

3.14 Harvested produce

`Assets/Scripts/TerrainPreview/Harvest`

Fruit that drops on the ground at harvest and is collected into the inventory.

CollectibleTropicalCrop.cs

The base component for harvested fruit lying on the ground: which crop, which item, and how many.

Key parts: `cropKind`, `itemType`, `amount`

`CollectibeMango.cs` · `CollectibleCacao.cs` · `CollectibleCorn.cs` · `CollectibleDurian.cs` · `CollectibleEggplant.cs` · `CollectibleMangosteen.cs` · `CollectiblePineapple.cs` · `CollectiblePomelo.cs` · `CollectibleSquash.cs` · `CollectibleStrawberry.cs` · `CollectibleTomato.cs`

Eleven subclasses that set their crop and item. ("CollectibeMango.cs" is the file's real, misspelled name.)

Key parts: `Awake`, `Reset`

`CollectibeCoconut.cs` · `CollectibleBanana.cs`

The same for coconuts and banana bunches, which have their own simulation classes.

Key parts: `amount`

3.15 Inventory

Assets/Scripts/TerrainPreview/Inventory

InventoryItemType.cs CHANGED · planting

The enum of every item: tools, planting materials, produce, liquids and kits. The 8 new planting materials were added at the end, so the numbers of existing items never change.

Key parts: `InventoryItemType`

InventorySlotUI.cs

One inventory slot: icon, count, tap to select, and drag and drop between slots.

Key parts: `Setup`, `OnPointerClick`, `OnDrop`

InventoryTrashUI.cs

Discard mode for the backpack, with an amount board and a confirmation.

Key parts: `Begin`, `Hide`

InventoryUIBuilder.cs CHANGED · planting

Builds the money display, the hotbar and the backpack, and draws slot icons. Icons for the 8 new items were added.

Key parts: `GetSpriteFor`, `ToggleBackpack`, `Refresh`

PlayerInventory.cs CHANGED · planting

The inventory data: slots, the selected item, money, stack limits and sprayer liquid. It adds, consumes and moves items, builds the starting inventory, and saves and restores itself. New: old seeds are turned into the new materials whenever they are added or loaded, and the full starting kit includes 3 Mulch Bags.

Key parts: `AddItem`, `ConsumeItem`, `MoveOrSwapSlots`, `CaptureSaveData`, `RestoreSaveData`, `GetMaxStack`

SprayerLiquidType.cs

The enum of sprayer liquids.

Key parts: `SprayerLiquidType`

3.16 Farm map and boundary

Assets/Scripts/TerrainPreview/Map

FarmMapFieldInput.cs

Pinch, scroll and drag input on the expanded map, passed to the map builder.

Key parts: `OnDrag`, `OnScroll`

FarmMapIcons.cs

Chooses and shrinks the pictures the map draws: a crop's fruit, or a structure's kit.

Key parts: ForCrop, ForClimateObject

FarmMapUIBuilder.cs CHANGED · planting

The farm map. A small minimap sits under the money plank and turns with the player. Tapping it opens a north-up map of the whole farm with zoom and pan, showing markers for the player, crops, the bin, structures and traps. A Seedling Tent marker was added.

Key parts: SetExpanded, RebuildMarkers, ZoomAtScreenPoint

MapBoundarySystem.cs

Invisible walls around the terrain edge, plus a soft fog curtain that hides the edge.

Key parts: BuildWalls, BuildFog

3.17 Seedling Tent and planting materials (new folder)

Assets/Scripts/TerrainPreview/Nursery

The heart of the new planting system.

PlantingMaterialCatalog.cs NEW · planting

The 16 planting materials and the route each takes to the field: bag, direct or ready seedling, the kind of ground, whether mulch is needed, nursery days, prick-out day, extra age, and real-world text. It also upgrades the five retired seed names, works out the starting age, and formats the crop board lines (CropPlantingText). Every other system asks this file instead of hard-coding crops.

Key parts: TryGet, PlotFor, NeedsMulchedBed, UpgradeLegacy, StartingAgeDays, SeedlingName, CropPlantingText.PlantedFromLine

NurserySystem.cs NEW · planting

The logic of the Seedling Tent's 8 bags. It works out each bag's status (Empty, Filled, Germinating, NeedsPricking, Growing, Ready) from the game clock, and handles fill, sow, prick out, empty, and begin, cancel or finish transplant. It places the tent on a clear spot near the player, saves and restores the bags, and reports nursery actions to the task hub.

Key parts: GetStatus, DaysUntilNextStep, FillBag, Sow, PrickOut, BeginTransplant, FinishTransplant, CaptureSaveData, RestoreSaveData

SeedlingTentInstance.cs NEW · planting

The tent standing on the farm. It draws each bag, empty or filled, and each seedling growing out of it, scaled by progress and with its roots hidden in the soil. It rests the tent on the terrain, and tapping it opens the panel.

Key parts: SeatOnTerrain, Refresh, RefreshGrowth

SeedlingTentUIBuilder.cs NEW · planting

The tent panel and its HUD button (slot 7). It shows a grid of the 8 bags, a detail area with the right action buttons, a sow picker, a description board, and a banner while a seedling is being carried.

Key parts: OpenPanel, Redraw, RebuildActions, OpenSowPicker, OnTransplant, RefreshBanner

PlantingAvailability.cs NEW · planting

Answers "what could be planted today?" for the daily tasks and the AI task check. It counts materials that can go in directly, bought seedlings, and ready tent seedlings, but only where ground can be prepared for them.

Key parts: CountPlantableToday, CanPrepareGroundFor, CountFreeBags

NurseryRuntimeBootstrap.cs NEW · planting

Creates `NurserySystem` and `SeedlingTentUIBuilder` when the farm scene loads, like the other runtime systems.

Key parts: `EnsureInstalled`

3.18 Saving and loading the farm

Assets/Scripts/TerrainPreview/Persistence

FarmPersistenceManager.cs CHANGED · planting

Saves and loads the farm. `SaveFarm` captures a snapshot and sends `PUT /api/farms/me` with the revision it started from, and the server answers 409 if another device saved first. After the terrain is generated it restores in a fixed order: tutorial, time, weather, inventory, player, crops, structures, prepared ground, tent, climate event, tasks. Now saves schema version 4, including the tent and unplanted plots.

Key parts: `SaveFarm`, `CaptureSnapshot`, `RestoreGameState`, `CurrentSchemaVersion`

FarmSnapshotModels.cs CHANGED · planting

All the save data classes: `FarmSnapshotDto` with area, soil, player, inventory, time, weather, climate event, tasks, crops, structures and tutorial. `FarmLoadContext` holds the snapshot while it is being loaded. New:

`NurserySaveDto`, `SeedlingBagSaveDto`, `PreparedPlotSaveDto`, and planting fields on each crop.

Key parts: `FarmSnapshotDto`, `CropSaveDto`, `NurserySaveDto`, `PreparedPlotSaveDto`, `FarmLoadContext`

CropPersistenceMapper.cs CHANGED · planting

Converts each crop to and from its save data: common fields, diseases, maintenance and protections. The planting fields were added, and fruit bags and drainage kits are now saved (a bug fix).

Key parts: `Capture`, `Restore`, `RestoreProtection`

SaveFarmButtonBuilder.cs

The Save Farm button on the HUD, with a confirmation.

Key parts: `Build`, `ShowConfirmation`

3.19 Pests and diseases

Assets/Scripts/TerrainPreview/PestDisease

A data-driven pest and disease simulation covering 30 conditions across the 13 crops.

PestDiseaseSystem.cs

Rolls the daily risk for every crop and every rule, weighing month, growth stage, temperature, humidity, weather, plant stress, soil moisture and nearby infections. It starts or worsens conditions, simulates their progress, and prepares a pest summary for the adviser.

Key parts: `ProcessNewGameDay`, `EvaluateRuleForCrop`, `SimulateDiseaseProgress`, `Force*`

PestDiseaseAffectedCrop.cs

The pest and disease state of one crop: active conditions, their severity growth, and the damage to health and stress. It also applies treatments (sprays, disinfectant, removal).

Key parts: `StartOrIncreaseCondition`, `ApplyMitigation`, `ReduceCondition`

PestDiseaseRule.cs

One rule: the crop, peak months, stages, temperature and humidity limits, weather preferences, daily chance, severity growth, damage, spread and the treatments that work.

Key parts: `PestDiseaseRule`, `CropDevelopmentStage`

PestDiseaseDatabase.cs

The ScriptableObject asset that holds all the rules.

Key parts: `GetRulesForCrop`, `GetRule`

PestDiseaseType.cs

Enums for the 30 pest and disease types (for example aphids, fall armyworm, bacterial wilt, Panama disease, bugtok, black pod rot), their categories, and the mitigations.

Key parts: `PestDiseaseType`, `PestDiseaseCategory`, `PestDiseaseMitigation`

ActivePestDisease.cs

One active condition on a crop: type, severity, target severity and start day.

Key parts: `fields`

AphidTrapInstance.cs

A placed aphid trap. It drains aphids from nearby crops until it is full, and can be cleaned.

Key parts: `CleanTrap`, `SimulateTimeSkip`

AreaMitigationTrapInstance.cs

Pheromone and bait traps and termite bait stations. Each applies its mitigation to crops within its radius.

Key parts: `ApplyAreaEffect`, `CleanTrap`

PestDiseasePopupUI.cs

The popup shown when a pest or disease appears.

Key parts: `Show`, `Hide`

3.20 Player control

Assets/Scripts/TerrainPreview/Player

FirstPersonTerrainController.cs

Movement, jumping, gravity and camera look, by mouse or touch. It sends taps to the farming and climate systems, ignoring taps on the UI.

Key parts: `HandleMovement`, `HandleMobileTouchLookAndTap`, `SnapToTerrain`

FreeCamController.cs

A free-flying camera used during development.

Key parts: `HandleLook`, `HandleMove`

MobileHudBuilder.cs

The on-screen joystick and jump button on phones.

Key parts: `MoveInput`, `JumpPressCount`

VirtualJoystick.cs

The joystick control itself.

Key parts: OnDrag, OnPointerUp

3.21 Friends, chat, marketplace and trades

Assets/Scripts/TerrainPreview/SocialMarketplace

SocialMarketplaceApi.cs

Every social and economy HTTP call: presence heartbeat, player search, friends and requests, chat, marketplace listings, fees, buying and sale notices, and all trade steps.

Key parts: Heartbeat, SearchPlayers, SendMessage, CreateListing, BuyListing, InviteTrade, ConfirmTrade

SocialMarketplaceController.cs

Background loops for the presence heartbeat, notifications (friend requests, unread messages, sales, admin commands) and the active trade. It saves before every economy action, and afterwards merges the server's result into the live game. It also creates, buys and cancels listings and runs the trade steps.

Key parts: SaveBeforeEconomy, SyncRemoteEconomyPreservingLocalChanges, CreateListing, BuyListing, ConfirmTrade

EconomyRevisionMerge.cs CHANGED · planting

A three-way merge of money and items between the last synced state, the local state now, and the server state now. Changes made on the server by a sale or trade are applied without losing anything the player did locally. It now also covers the retired seed items.

Key parts: Clone, TryMerge

SocialMarketplaceCatalog.cs CHANGED · planting

Which items can be listed or traded and their base values in centavos, used for listing fees, plus friendly names. It must match the server. The new materials were added, and MergeItems covers old seeds too.

Key parts: IsTradable, GetBaseValueCentavos, CalculateListingFee, FriendlyName

SocialMarketplaceDtos.cs

DTOs for the social, marketplace and trade JSON.

Key parts: MarketplaceListingDto, TradeViewDto

SocialMarketplaceRuntimeBootstrap.cs

Creates the social systems in the farm scene.

Key parts: Register

SocialMarketplaceUIBuilder.Core.cs

The shared UI parts of the social screens: top buttons, notification badges and panel helpers.

Key parts: BuildTopButtons, CreatePanel

SocialMarketplaceUIBuilder.Marketplace.cs CHANGED · planting

The marketplace panel: item filter, listing rows, the sell panel and sale notifications. It shows friendly names for the new materials.

Key parts: RefreshMarketplace, OpenSellPanel, BuyListing

SocialMarketplaceUIBuilder.PlayersChat.cs

Player search, profiles, friend requests and the chat window, which polls for new messages.

Key parts: SearchPlayers, OpenPlayerProfile, ChatPollLoop

SocialMarketplaceUIBuilder.Trade.cs

The live trade screen. Players drag items into offer slots and set money, then press Agree and Confirm. A notice appears when a seed is not grown in the partner's district.

Key parts: ShowTradePanel, PushOffer, OnFinalTradeConfirmPressed

TradeDragSource.cs

The drag source (an inventory item) and the drop slot (a 2 × 2 offer cell) for the trade screen.

Key parts: TradeDragSource, TradeDropSlot

3.22 Soil

Assets/Scripts/TerrainPreview/Soil

SoilGridsService.cs

Queries the ISRIC SoilGrids v2.0 API for sand, silt, clay, pH, organic carbon, coarse fragments, bulk density and nitrogen at 0–5 cm. If the request fails it uses a sensible default sample.

Key parts: QuerySoil

SoilGridsJsonParser.cs

Reads the SoilGrids JSON reply and converts it into normal units.

Key parts: Parse

SoilGameplayRules.cs

Turns soil numbers into gameplay values. Drainage rises with sand and falls with clay and compaction. Fertility weighs nitrogen at 45%, organic carbon at 35% and a pH close to 6.5 at 20%. There is also a compaction penalty.

Key parts: GetDrainage, GetFertility, GetCompactionPenalty

SoilSample.cs

The soil numbers, plus a visual soil type such as Loamy.

Key parts: GetVisualSoilType

SoilInfoPanelBuilder.cs

The scrollable crop information readout on the left of the HUD, with a field-guide button.

Key parts: ShowInfo, HideInfo

3.23 Terrain and world

Assets/Scripts/TerrainPreview/Terrain

SoilAwareTerrainGenerator.cs

Runs world generation. It builds the terrain, places the player, and gets the two soil samples (or restores the saved ones). It blends them into a soil grid across the farm, adds the ground colour and grass texture, and produces the crop information text.

Key parts: Start, BuildSoilGrid, TryGetSoilAtWorldPosition, ShowFullCropInfo

TemporaryTerrainGenerator.cs

Builds the 700 × 700 m heightmap for the selected area from the official Davao elevation and slope maps. It matches legend colours to heights, smooths the result, and adds natural noise and small hills in flat land.

Key parts: `Generate`, `BuildHeightsFromOfficialMaps`, `ResolveActiveTerrain`

DistanceCullable.cs

Marks an object whose renderers should hide when it is far away. Its scripts keep running while hidden.

Key parts: `Attach`, `SetVisible`

DistanceCullingManager.cs

Shows and hides registered objects by distance, using the player's render-distance setting, at a throttled rate.

Key parts: `Register`, `Unregister`

GrassPatchTerrainSpawner.cs

Scatters grass patches over the terrain and shows only the nearby ones.

Key parts: `SpawnGrassPatches`

TreeBillboardLOD.cs

Swaps distant tree models for flat pictures to save performance.

Key parts: `CheckDistance`, `SetBillboardMode`

3.24 Beginner tutorial (Antonio)

Assets/Scripts/TerrainPreview/Tutorial

TutorialDirector.cs CHANGED · planting

Runs the beginner guide as a flat list of beats: Antonio's lines, silent actions (reveal a button, give an item) and gates (wait for the player to do something). It gives the starting materials and saves at the end. Updated: Step 3 teaches till, prepare and plant, and only moves on for the right plot kind, with a live hint that explains how to recover. New Step 4B introduces the Seedling Tent.

Key parts: `BuildBeats`, `Advance`, `WaitForAction`, `Step3PrepareAndPlant`, `GroundHint`, `Step4BSeedlingTent`, `Finish`

TutorialDialogueUI.cs CHANGED · planting

Antonio's dialogue board (portrait videos, typed text, tap to continue) and the objective plank shown while the player acts. New: `SetObjectiveText` for live hints, and `ObjectiveShowing`, so the tent panel hides its heading behind the plank.

Key parts: `ShowLine`, `ShowObjective`, `SetObjectiveText`

TutorialGates.cs

The conditions the guide waits for: move and jump, a farm action (read from `FarmTaskActionHub`), a panel opened and closed, bought something, reward claimed, day skipped.

Key parts: `MoveAndJump`, `FarmAction`, `PanelOpenedThenClosed`

TutorialState.cs CHANGED · planting

Remembers the tutorial's progress and the farm's three starting materials. It is saved in the snapshot. Old seed names are upgraded on load.

Key parts: `Completed`, `StartingSeeds`, `Capture`, `Restore`

3.25 Farm popups and loading

Assets/Scripts/TerrainPreview/UI

FarmChoicePopup.cs NEW · planting

A board with a question and up to three answer planks plus Back. It asks "Planting Hole, Raised Bed or Furrow?" when tilled ground is tapped with the shovel.

Key parts: Show(message, choices), Choice

FarmConfirmPopup.cs

The shared Yes/No confirmation board.

Key parts: Show, Hide

WorldLoadingScreen.cs

The loading screen, with status lines, shown while the farm generates.

Key parts: Show, SetStatus, Hide

3.26 Time and weather

Assets/Scripts/TerrainPreview/Weather

GameTimeSystem.cs

The game clock: 900 s per day, 16 days per month, 12 months, starting at 08:00. It moves the sun, raises new-day events, and simulates skipped time in bounded steps.

Key parts: TotalGameDays, AdvanceDays, OnNewGameDay, RestoreTime

WeatherSystem.cs

Rolls each day's weather from monthly profiles of Davao's climate: temperature range, humidity, rainfall, rainy days, sunshine, and chances of rain, typhoon and extreme drought. It runs events with a duration and tells crops how fast they dry or gain water.

Key parts: AdvanceWeatherOneDay, ForceWeatherEvent, GetDryingMultiplier, OnWeatherChanged

WeatherVisualController.cs

The look of the weather: rain particles, storm sky, lightning, fog, ambient light and sun strength.

Key parts: OnWeatherChanged, ApplyStormSky, UpdateLightning

FarmGraceperiod.cs

Keeps the first two game days of a new farm calm: no rain, no climate events, no outbreaks.

Key parts: grace check

3.27 Editor tools (run inside Unity, not in the game)

Assets/Editor

Menu items and import rules that prepare assets. None of these run on the phone.

PlantingAssetsSetup.cs NEW · planting

Menu AgriDabao → Planting. Step 1 sets the new UI images up as sprites. Step 2 builds 21 prefabs from the Meshy models (seedlings, materials, bags, ground models and the tent with racks and bag anchors) at real-world scale. Step 3 wires them into the farm scene, the inventory icons and the UI theme.

Key parts: SetUpSprites, BuildPrefabs, WireScene

MeshDecimationPostprocessor.cs CHANGED · planting

Reduces the triangle count of imported models automatically while leaving the source files untouched, to meet the mobile budget. The 18 new model folders were added.

Key parts: `OnPostprocessModel`, `TryGetQuality`

MeshStatsReporter.cs

Lists every model's triangle count, heaviest first.

Key parts: `Report`

PestDiseaseDatabaseSeeder.cs

Fills the pest and disease database with the rules for all 13 crops.

Key parts: `PopulateSelectedDatabase`, `Add*Rules`

DistrictMapArtDrawer.cs

An Inspector drawer that labels each district's art row by name.

Key parts: `OnGUI`

DistrictMapArtPostprocessor.cs

Correct import settings for the map pictures. Maps that are read pixel by pixel are uncompressed and readable.

Key parts: `OnPreprocessTexture`

TutorialPictureImporter.cs

Imports the tutorial screenshots as sprites automatically.

Key parts: `OnPreprocessTexture`

ClimateMaintenance/ClimateMaintenanceAssignmentValidator.cs

Checks that every climate prefab and sprite is assigned.

Key parts: `ValidateAssignments`

ClimateMaintenance/ClimateMaintenancePrefabGenerator.cs

Generates simple prefabs for the structures and care visuals from basic shapes.

Key parts: `Generate`

3.28 Other project folders

Location	What it holds
<code>Assets/Scenes</code>	<code>MainMenu</code> , <code>AreaSelection</code> , <code>TerrainPreview</code> (the farm) and Unity's template <code>SampleScene</code> .
<code>Assets/Resources</code>	<code>UITheme.asset</code> (all UI sprites), fonts, shaders and climate-maintenance prefabs loaded by name at runtime.
<code>Assets/Models</code> / <code>Assets/Prefab</code>	3D models (FBX), including the Meshy planting models, and the prefabs built from them.
<code>Assets/Sprites</code>	UI art (<code>Sprites/UI</code>), item icons (<code>Sprites/CropFruitsSeeds</code>) and the Davao maps (<code>Sprites/DavaoCityMap</code>).
<code>Assets/Data</code>	Data assets such as the pest and disease database.
<code>Docs/</code> , <code>Handoff/</code>	The tutorial script, the data security assessment, the planting handoff notes and the crop notes the real-world figures come from.

Part 4 — Backend server: every file

67 Java source files in `com.agridabao.api`, plus one test, grouped by package.

The backend is one Spring Boot application in the package `com.agridabao.api`. It follows the usual Spring layering:

- A **controller** (`@RestController`) receives HTTP requests, validates the JSON body (`@Valid`) and reads the signed-in player from the JWT.
- A **service** (`@Service`) holds the rules and runs inside a database **transaction** (`@Transactional`), so a multi-step change either fully happens or not at all.
- An **entity** (`@Entity`) maps a Java class to a database table.
- A **repository** (`JpaRepository`) reads and writes that table, including locking queries (`SELECT ... FOR UPDATE`) where two requests could collide.

Several small feature areas (chat, marketplace, social, trade) keep their controller, records, entity, repository and service in one file.

4.1 Application entry point

`src/main/java/com/agridabao/api/` · `com.agridabao.api`

AgridabaoApiApplication.java

The `main` method that starts Spring Boot (`@SpringBootApplication`). Spring then finds every controller, service and configuration class in the package.

Key parts: `main`

4.2 Security, tokens and error handling

`src/main/java/com/agridabao/api/` · `config` / `security` / `error`

How requests are authenticated and how failures are reported.

config/SecurityConfig.java

Configures stateless security. `/api/auth/**`, `/actuator/health`, `/actuator/info` and `/error` are public; every other endpoint needs a valid JWT. Tokens are checked with a custom decoder: the HS256 signature and expiry, the `purpose` claim, which must be an access token (so a password-reset ticket cannot be used to log in), and the `ver` claim, which must equal the account's current token version (a password change ends older sessions). It also defines BCrypt with cost 12 and loads the signing key from `APP_JWT_SECRET_BASE64`, which must decode to at least 32 bytes.

Key parts: `securityFilterChain`, `accessTokenDecoder`, `requireAccessPurpose`, `requireCurrentTokenVersion`, `passwordEncoder`

config/SchedulingConfig.java

Turns on Spring's scheduler (`@EnableScheduling`) so timed jobs can run, such as returning expired marketplace listings every minute.

Key parts: `@EnableScheduling`

security/JwtService.java

Issues tokens. An access token carries the user ID, email, `purpose=access` and the token version, and lasts 30 days by default (`APP_ACCESS_TOKEN_MINUTES`). A password-reset ticket lasts 15 minutes and is only accepted by the reset endpoint.

Key parts: `issue`, `issueResetTicket`, `readResetTicket`

error/ApiExceptionHandler.java · error/ApiError.java

Turns exceptions into one uniform JSON error body (`ApiError`) with the right HTTP status: `BadRequest` → 400, `Unauthorized` → 401, `Forbidden` → 403, `NotFound` → 404, `Conflict` → 409, and validation failures → 400 with the field errors. The game's `ApiClient` reads the `message` field from this body.

Key parts: `@RestControllerAdvice`, `badRequest`, `conflict`, `validation`

error/BadRequestException.java · ConflictException.java · ForbiddenException.java · NotFoundException.java · UnauthorizedException.java

The five exception types services throw to reject a request. Each maps to one HTTP status in the handler above.

Key parts: `RuntimeException` subclasses

4.3 Sign-up, login and password recovery

`src/main/java/com/agridabao/api/ · auth`

Two-step authentication: a password, then a code emailed to the player.

AuthController.java

The public auth endpoints. Sign-up: `POST /register/request` (validate and email a code), then `/register/verify` (create the account and return a token). Login: `/login/request` (check the password and email a code), then `/login/verify` (return a token). Recovery: `/password/forgot/request`, `/verify` and `/reset`.

Key parts: `requestRegister`, `verifyRegister`, `requestLogin`, `verifyLogin`, `requestPasswordReset`, `verifyPasswordReset`, `resetPassword`

AuthService.java

The rules behind those endpoints. It normalises the email, checks that the passwords match and that the email and display name are free, hashes the password, creates and emails codes, verifies codes, creates the account and issues the token. A wrong password and an unknown account get the same message, so the endpoint cannot be used to discover accounts. Recovery shows only a masked email. A player can sign in with either their email or their display name.

Key parts: `requestLogin`, `verifyLogin`, `findByIdentifier`, `buildResponse`

VerificationService.java

Creates and checks the emailed codes. Codes are 6 digits from `SecureRandom` and stored only as a BCrypt hash. They expire after 10 minutes, allow 5 wrong attempts and have a 30-second resend cooldown, and only one live code exists per purpose. Checks of the same code lock its row, so guesses sent at the same time are still counted one by one. All the limits can be changed through environment variables.

Key parts: `createLoginCode`, `consumeLoginCode`, `validateAndConsume`, `enforceResendCooldown`, `lockLive`

EmailVerification.java · EmailVerificationRepository.java · VerificationPurpose.java

The `email_verification` table: email, purpose (`SIGNUP`, `LOGIN`, `PASSWORD_RESET`, `EMAIL_CHANGE`), code hash, attempts, expiry, consumed time, and the pending sign-up details held until the code is confirmed. The repository includes locking lookups.

Key parts: `registerFailedAttempt`, `markConsumed`, `findLocked...`

**RegisterRequest · LoginRequest · VerifyCodeRequest · ForgotPasswordRequest ·
ForgotPasswordVerifyRequest · ResetPasswordRequest (records)**

The request bodies, validated with annotations. For example, sign-up requires a valid email, a password of 8–72 characters, a confirmation and an optional display name of up to 80 characters. The login identifier may be an email or a display name.

Key parts: @NotBlank, @Email, @Size

AuthResponse · CodeRequestResponse · PasswordResetTicketResponse · UserResponse (records)

The response bodies. `AuthResponse` carries the token, its expiry, whether a farm exists, and the user.

`CodeRequestResponse` includes the code itself only in development (`APP_VERIFICATION_EXPOSE_CODE`), never in production.

Key parts: records

4.4 The account

src/main/java/com/agridabao/api/ · user

AppUser.java · AppUserRepository.java

The `app_user` table (id, email, password hash, display name, date of birth, token version, timestamps) and its queries: find by email or by display name ignoring case, search by name, and existence checks. Changing the password increments the token version.

Key parts: `changeEmail`, `changePasswordHash`, `changeDisplayName`

UserController.java

Endpoints for the signed-in player: GET `/api/users/me`, email change (`/me/email/request` then `/me/email/verify`), `/me/password` and `/me/display-name`.

Key parts: `me`, `requestEmailChange`, `confirmEmailChange`, `changePassword`, `changeDisplayName`

AccountService.java

The logic for those changes. The email change sends a code to the new address and the address is read back from the server's own record, so a code cannot confirm a different address. A password change requires the current password. It ends every other session and hands this phone a fresh token so it stays signed in. A display name change checks that the name is not taken.

Key parts: `requestEmailChange`, `confirmEmailChange`, `changePassword`, `changeDisplayName`

AccountFields.java

The shared rules for emails and display names: normalising, 3–80 characters for a name, and masking an email for display (for example `ab****@gmail.com`: the first two letters, then a fixed run of stars so the length is not revealed).

Key parts: `normalizeEmail`, `validateDisplayName`, `mask`

**ChangeDisplayNameRequest · ChangeEmailRequest · ChangePasswordRequest ·
ConfirmEmailChangeRequest (records)**

The request bodies for the account changes.

Key parts: records

4.5 Farm saves and the economy

src/main/java/com/agridabao/api/ · farm

Stores each player's farm and lets the marketplace and trades edit it safely.

FarmController.java

/api/farms/me: GET loads the farm (404 if none yet), PUT saves it, POST /logout records the logout time, and DELETE removes it.

Key parts: get, replace, logout, delete

FarmService.java

Saving with optimistic concurrency. The save must send expectedRevision equal to the stored revision (0 for a first save), or the server answers 409 Conflict. The row is locked while it is compared and written, and each successful save increases the revision by one. This stops an older copy from another device overwriting newer progress.

Key parts: replace, recordLogout

FarmSave.java · FarmSaveRepository.java

The farm_save table, one row per player: revision, schema version, generator version, the whole farm snapshot as PostgreSQL JSONB, the saved time and the logout time. findForUpdate reads the row with a lock.

Key parts: replace, findForUpdate

FarmSaveRequest.java · FarmSaveResponse.java

The save body (expectedRevision ≥ 0, schemaVersion ≥ 1, generatorVersion, snapshot) and the response. The snapshot is kept as a raw JSON tree, so fields the server does not know, such as the new tent and plots, are stored untouched.

Key parts: records

EconomyJsonService.java CHANGED · planting

Edits a farm's saved JSON on the server for the marketplace and trades. It reads and sets money, counts, removes and adds items while respecting stack limits, checks whether an item is tradable, returns its base value in centavos, and checks whether an item is allowed in a district. It works on a deep copy and bumps the revision on every change. Updated: the new planting materials are tradable with base values, and the Aphid Trap stack limit (99) now matches the game.

Key parts: editableSnapshot, saveChangedSnapshot, removeItem, canAddItem, addItem, isItemAvailableInDistrict, baseValueCentavos

DistrictSeedPools.java CHANGED · planting

The server's copy of which crops each district grows. A marketplace purchase or trade that would give a farm a planting material its district does not grow is refused here, so a modified or old client cannot get around the rule. Rewritten around crops: every material of an allowed crop is allowed, and the five retired seed names are still recognised for older game versions.

Key parts: isRestricted, isAvailableIn, cropsFor, poolFor

PesoRounding.java

Converts centavo totals to whole pesos, rounding halves up, exactly like the game's PesoPrice, so the listing fee shown equals the fee charged.

Key parts: toWholePesos

WeatherMitigationTradableItems.java

Adds the weather-mitigation kits to the tradable items, with their base values.

Key parts: isTradable, baseValueOrNull

4.6 Marketplace

src/main/java/com/agridabao/api/ · marketplace

MarketplaceController.java

/api/marketplace: GET /listings, GET /fee, POST /listings (create), POST /listings/{id}/buy, POST /listings/{id}/cancel (cancel or claim a return), GET /sales/unseen and POST /sales/{id}/acknowledge. The same file holds the marketplace_listing entity (status OPEN, SOLD, CANCELLED, EXPIRED or RETURN_PENDING; a listing expires after 3 days), its repository, MarketplaceService, and MarketplaceExpiryJob, which returns expired items every 60 seconds.

Creating a listing takes the items and a fee out of the seller's saved farm. The fee is the items' base value plus the asking price, paid up front. **Buying** locks both farms in a fixed order to avoid deadlocks. It checks the district, money and inventory space, then moves the money and items inside both saved farms in one transaction. Listings of planting materials that the viewer's district does not grow are hidden from that viewer.

Key parts: fee, create, buy, cancelOrClaim, expireDueListings, unseenSales

4.7 Player-to-player trades

src/main/java/com/agridabao/api/ · trade

TradeController.java

/api/trades: GET /active, /incoming, /{id}, POST /invite, /{id}/accept, /{id}/decline, PUT /{id}/offer, POST /{id}/agree, /{id}/confirm and /{id}/cancel. It holds the trade_session entity (PENDING, ACTIVE, COMPLETED, DECLINED, CANCELLED or EXPIRED; a session expires after 15 minutes) and TradeService.

The rules: both players must be online, and each may have only one open trade. Every offer is checked against the offerer's saved farm, and planting materials against the receiver's district, again at Agree and again at the exchange. The exchange happens only after both players Agree and then both Confirm. It locks both farms in a fixed order and swaps items and money in both saved snapshots inside one transaction.

Key parts: invite, updateOffer, agree, confirm, executeExchange, requireSeedsAvailableIn

4.8 Friends, presence and chat

src/main/java/com/agridabao/api/ · social / chat

social/SocialController.java

/api/players/search (up to 30 results), /api/players/{id}, /api/friends, /api/friends/requests, and sending, accepting or declining a friend request. It holds the friend_request and friendship entities and works out the relationship (self, none, pending either way, friends).

Key parts: search, profile, sendRequest, accept, decline

social/PresenceController.java

POST /api/presence/heartbeat and /offline. It keeps one active device per account: a device may claim the account if nobody holds it or the holder has been silent for more than 45 seconds. Otherwise it answers 409 with a message for the second phone. "Online" means seen within those 45 seconds.

Key parts: heartbeat, markOffline, isOnline, ONLINE_WINDOW

social/PresenceAccessService.java · social/FriendshipAccessService.java ·

social/FriendshipId.java

Small helpers other features use: "is this player online?" (needed for trades) and "are these two friends?" (needed for chat). FriendshipId is the two-column key of a friendship.

Key parts: isOnline, areFriends

chat/ChatController.java

`/api/chat/{otherPlayerId}/messages` (GET with an optional `after` time for polling, POST to send up to 1,000 characters) and `/api/chat/unread-count`. Only friends can message each other. It holds the `chat_message` entity and marks messages read.

Key parts: `messages`, `send`, `unread`, `requireFriend`

4.9 AI proxy for the farm adviser

src/main/java/com/agridabao/api/ · ai

The only route from the game to Google Gemini.

AiController.java

GET `/api/ai/status` and POST `/api/ai/generate`. `AiService` picks the generation settings per feature: ADVISOR (temperature 0.4, top-p 0.9, up to 3,000 tokens), CLIMATE_EVALUATION (no cap), and TASK_GENERATE and TASK_CHECK (temperature 0.25, JSON output). It applies the guardrails, limits the player's question to 1,000 characters and quotes it. If the AI is switched off (`APP_AI_ENABLED`) or fails, it replies in Antonio's voice that he is busy rather than returning an error.

Key parts: `generate`, `configFor`, `prepareParts`

AiGuardrails.java

The adviser's rules, enforced on the server because the game's prompt ships inside the APK and could be altered. They are placed above the game's own instructions: answer only farming and game questions; reply to anything else with one fixed line; treat the player's text as data, not instructions; never claim to run code or reach data; give no harmful advice; never reveal the instructions.

Key parts: `systemInstructionFor`, `fencePlayerQuestion`, `REFUSAL`

GeminiClient.java

Calls Google's Generative Language API (`models/{model}:generateContent`) with the key from `GEMINI_API_KEY` and the model from `GEMINI_MODEL` (default `gemini-2.5-flash`). It uses a 15-second connect timeout and a 60-second request timeout, and asks for JSON output for the task features.

Key parts: `generate`, `isConfigured`, `GenerationConfig`

4.10 Player settings

src/main/java/com/agridabao/api/ · settings

PlayerSettingsController.java · PlayerSettingsService.java · PlayerSettings.java · PlayerSettingsRepository.java · PlayerSettingsRequest.java · PlayerSettingsResponse.java

GET and PUT `/api/settings/me`: the per-account settings (music, SFX and ambience volume, render distance, interface and text size, AI summarisation) in the `player_settings` table. Values are clamped to safe ranges. A missing size value from an older game keeps the stored value instead of shrinking the UI.

Key parts: `get`, `upsert`, `clamp`

4.11 Email

src/main/java/com/agridabao/api/ · mail

MailService.java

Sends the code emails (sign-up, login, password reset, email change) from the HTML templates in `resources/email`. It uses Resend when `RESEND_API_KEY` is set, which is required on Railway because cloud hosts block SMTP, and Gmail SMTP otherwise.

Key parts: `sendSignupCode`, `sendLoginCode`, `sendPasswordResetCode`, `sendEmailChangeCode`

ResendMailClient.java

The client for Resend's HTTPS email API (port 443, which is never blocked).

Key parts: `send`, `isEnabled`

4.12 Developer and admin tools

`src/main/java/com/agridabao/api/` · `admin`

AdminController.java

`/api/admin: /capability` (may this account use the tools, and on a phone?), `/accounts/lookup`, `/accounts/delete` (removes a test account and all of its data), `/commands` (queue money, a weather event or a pest event for a player) and `/commands/pending` (the game collects its queued commands). Only emails listed in `APP_ADMIN_EMAILS` are allowed, and an empty list means nobody. Phone access is a separate switch (`APP_ADMIN_MOBILE_TOOLS`).

Key parts: `capability`, `delete`, `sendCommand`, `pendingCommands`

AdminCommand.java

The `admin_command` table: one queued instruction (`GRANT_MONEY`, `FORCE_WEATHER` or `FORCE_PEST_DISEASE`) for one player. Rows are kept after delivery as a record of what was sent.

Key parts: `markDelivered`

4.13 Tests

`src/main/java/com/agridabao/api/` · `src/test`

AgridabaoApiApplicationTests.java

A Spring Boot test that checks the application context starts.

Key parts: `contextLoads`

Part 5 — Database, configuration and deployment

5.1 Database tables (Flyway migrations V1–V11)

Table	Created in	What one row is
<code>app_user</code>	V1 (+V9, V10)	An account: email, BCrypt password hash, display name (unique ignoring case, since V9), date of birth, token version (V10).
<code>farm_save</code>	V1	A player's farm: revision, schema version, generator version, the snapshot as JSONB, saved time, logout time. Deleted with the account (<code>ON DELETE CASCADE</code>).
<code>player_presence</code>	V2 (+V8)	Who is online, and on which device (<code>device_id</code> , V8).
<code>friend_request</code> , <code>friendship</code>	V2	Friend requests (PENDING, ACCEPTED, DECLINED) and friendships stored once per pair (lower ID, higher ID).
<code>chat_message</code>	V2	One message between two players (up to 1,000 characters) with sent and read times.
<code>marketplace_listing</code>	V2 (+V5)	A listing: seller, item, quantity, asking price, fee, status, expiry, buyer, and whether the seller has seen the sale (V5).
<code>trade_session</code>	V2	A trade between two players: status, each side's offer as JSONB, and agreed and confirmed flags.
<code>email_verification</code>	V3 (+V9)	A pending emailed code: purpose, code hash, attempts, expiry, pending sign-up details, and <code>user_id</code> for email changes (V9).
<code>player_settings</code>	V4 (+V6, V7)	Volumes and render distance, interface and text size (V6), and AI summarisation (V7).
<code>admin_command</code>	V11	A queued developer command for one player.

Flyway runs the numbered scripts in `src/main/resources/db/migration` once each, in order, at start-up, and records which have run. Hibernate is set to `validate`, so it never changes the schema itself. A failed migration stops the server from starting, which is why migrations are written with extra care.

5.2 Configuration

Setting (environment variable)	Meaning
<code>PORT</code>	Port to listen on. Railway provides it; the default is 8080.
<code>SPRING_DATASOURCE_URL</code> , <code>_USERNAME</code> , <code>_PASSWORD</code>	The PostgreSQL connection.
<code>APP_JWT_SECRET_BASE64</code>	The signing key for tokens (Base64, at least 32 bytes). There is no default: the server refuses to start without it.
<code>APP_ACCESS_TOKEN_MINUTES</code>	Access token lifetime. The default of 43,200 minutes is 30 days.
<code>GEMINI_API_KEY</code> , <code>GEMINI_MODEL</code> , <code>APP_AI_ENABLED</code>	The AI key, the model (default <code>gemini-2.5-flash</code>) and the master on/off switch.
<code>RESEND_API_KEY</code> , <code>MAIL_*</code>	Email through Resend, or SMTP settings for local runs.
<code>APP_VERIFICATION_*</code>	Code length (6), lifetime (10 min), attempts (5), resend cooldown (30 s), and whether to expose codes (development only).
<code>APP_ADMIN_EMAILS</code> , <code>APP_ADMIN_MOBILE_TOOLS</code>	Who may use the admin endpoints, and whether they may do so from a phone.

No secret is written in the code or in `application.yml`. Every secret comes from an environment variable on Railway.

5.3 Build and deployment files

File	Purpose
<code>build.gradle</code> , <code>settings.gradle</code> , <code>gradlew</code>	The Gradle build: Java 21 toolchain, Spring Boot 4.1.0, and the starters for web, mail, security, OAuth2 resource server, JPA, validation, actuator and Flyway, plus the PostgreSQL driver.
<code>Dockerfile</code>	Two stages. The first builds the jar with Gradle on <code>eclipse-temurin:21-jdk</code> ; the second runs it on the smaller <code>eclipse-temurin:21-jre</code> image with no source code. Railway builds this image whenever a commit lands on GitHub <code>main</code> .
<code>src/main/resources/application.yml</code>	All configuration, reading the environment variables above. It exposes only <code>/actuator/health</code> and <code>/actuator/info</code> , and turns off the mail health check that would falsely report the service as down on cloud hosts.
<code>src/main/resources/email/*.html</code>	The HTML templates for the four code emails, plus the header image.
<code>scripts/*.ps1</code> , <code>verify-database.sql</code>	PowerShell scripts to run the backend locally and to exercise the API (backend, social, task persistence and weather-mitigation trading), and a SQL check of the database.
<code>LICENSE</code> , <code>THIRD-PARTY-NOTICES.md</code> , <code>HELP.md</code>	The project's copyright notice, the licences of the libraries used, and Spring's generated help file.

Part 6 — Questions a panel may ask

Short answers, with the files that back them up.

Where is the Gemini API key, and can someone steal it from the APK?

It is only on the server, in the `GEMINI_API_KEY` environment variable on Railway. The game sends prompts to `/api/ai/generate` with the player's JWT, and `GeminiClient` calls Google. Earlier builds held the key on `MonoBehaviours`, which ship inside the APK. Files: `AiBackendClient.cs`, `AiController.java`, `GeminiClient.java`.

How do you stop players from misusing the AI (prompt injection)?

`AiGuardrails.java` puts the server's rules above whatever instructions the phone sends: farming topics only, one fixed refusal line, the player's text treated as data, no code or SQL, no harmful advice, no leaking the instructions. The player's question is fenced (quoted) and limited to 1,000 characters (`AIService.prepareParts`).

How are passwords and login codes stored?

Passwords are hashed with BCrypt at cost 12 (`SecurityConfig.passwordEncoder`). Emailed codes are 6 random digits, stored as BCrypt hashes. They expire after 10 minutes, allow 5 wrong attempts and have a 30-second resend cooldown (`VerificationService`).

What is the JWT and what is inside it?

It is an HS256-signed token issued by `JwtService` containing the user's ID (subject), email, a purpose claim (`access`) and the account's token version. The server checks the signature, the expiry (30 days by default), the purpose (so a password-reset ticket cannot be used as a login) and that the token version still matches the account. Changing the password raises the version and ends older sessions.

What stops two phones from overwriting the same farm?

Two things work together. First, each save sends `expectedRevision`: `FarmService` locks the row and answers 409 if the stored revision differs, so a stale device cannot overwrite a newer save. Second, the presence heartbeat allows only one device per account at a time (`PresenceController`, 45-second window).

How are trades and marketplace purchases kept fair?

Everything that moves items or money happens on the server, inside a database transaction (`MarketplaceController`, `TradeController`, `EconomyJsonService`). The server checks money, stock and inventory space against the saved farm, applies district crop rules (`DistrictSeedPools`) and locks both farms in a fixed order. Trades also require both players to be online and to press Agree, then Confirm.

Where does the soil data come from?

From the ISRIC SoilGrids v2.0 REST API. `SoilGridsService` queries two points inside the selected area (25% and 75% across the box, `DavaoGeoReference`). `SoilAwareTerrainGenerator` blends them into a soil grid over the terrain. If the service cannot be reached, a default sample is used so the game still runs.

How is the terrain made?

`TemporaryTerrainGenerator` reads the official Davao elevation and slope maps (images) for the selected rectangle. It matches their legend colours to heights, smooths the result, adds natural noise and small hills in flat areas, and builds a 700 × 700 m Unity terrain.

How is the climate response graded after a typhoon or drought?

`ClimateEventTracker` snapshots every crop at the start and end and records every action taken in between. It computes a 0–100 score: a base of 45, plus each action's effectiveness, plus the average change in health, stress and moisture, minus 15 if the player did nothing. It then asks Gemini for a written evaluation.

Why are nursery times only a few days?

One game day is 15 real minutes. A real cacao nursery takes 5–12 months, which would be many hours of play. The game compresses the wait and shows the real figure on the crop board (`PlantingMaterialCatalog.RealWorld`).

Why is all the UI created in code?

It keeps every screen in version-controlled C#, with no prefabs to fall out of date. Art slots live in one asset (`UIThemeSprites`). Services such as `UIScaleService` (interface and text size) and `GameAccessibility` (TalkBack) can then find and adjust every panel automatically.

How is mobile performance handled?

The game uses several techniques:

- distance culling of renderers (`DistanceCullingManager`) and billboards for far trees (`TreeBillboardLOD`);
- automatic mesh decimation on import (`MeshDecimationPostprocessor`);
- a fixed frame-rate cap (`MobilePerformanceBootstrap`);
- on-screen measuring tools for the non-functional requirements (`FpsCounterHud` for NFR-01, `UiResponseTimerHud` for NFR-02).

What database do you use and how does its structure change safely?

PostgreSQL. The schema is created and changed only by Flyway migrations `V1` to `V11`, applied automatically at start-up, and Hibernate only validates that the tables match (`ddl-auto: validate`). The farm itself is one JSONB document per player, so new game features rarely need a migration.

How is the backend deployed?

A two-stage Dockerfile: build the jar with Gradle on JDK 21, then run it on a JRE 21 image. Railway rebuilds and redeploys automatically when a commit lands on GitHub `main`. `/actuator/health` is the health check.